

SOFT VET

1

SOFT VET

Sistema de gestión para
veterinarias

Versión 1.0.0

Tecnicatura Universitaria en Programación

Trabajo Final Integrador 2025

Fecha de presentación: Mayo 2026

Tutor académico: Ing. Raúl Prado

Integrantes:

David Valdez Gramajo	– Leg: 61181
Mariano Celiz	– Leg: 61134
Ángel Pastrana	– Leg: 61105
Joaquín Díaz	– Leg: 61726



2. Contenido

2. Contenido.....	2
3. RESUMEN EJECUTIVO.....	6
3.1 Breve descripción del proyecto.....	6
3.2 Objetivos principales.....	6
3.3 Metodología utilizada	6
3.4 Resultados esperados	7
3.5 Conclusiones generales.....	7
4. INTRODUCCIÓN	7
4.1 Contexto y antecedentes	7
4.2 Justificación	8
4.3 Objetivo General	8
4.4 Objetivos Específicos.....	8
5. ESTADO DEL ARTE	9
5.1 Contexto tecnológico actual	9
5.2 Sistemas existentes y comparativa	9
5.3 Enfoques tecnológicos actuales	9
5.4 Modelado de datos y normalización	10
5.5 Seguridad y control de acceso en aplicaciones web	10
5.6 Fundamentación del aporte del proyecto.....	10
5.7 Conclusión del Estado del Arte.....	10
6. DESCRIPCIÓN DEL PROYECTO.....	11
6.1 Problemática Detectada.....	11
6.2 Solución Propuesta.....	11
6.3 Alcance del Proyecto.....	11
6.4 Análisis del Mercado	12
6.5 Criterios de Éxito	12
6.6 DIAGRAMA ENTIDAD-RELACIÓN	12
.....	13
6.7 Reglas de Negocio	14
6.7.1 Gestión de Pacientes e Historias Clínicas.....	14
6.7.2 Turnos y Peluquería.....	14
6.7.3 Productos y Stock.....	14
6.7.4 Ventas y Cobros.....	14
6.7.5 Seguridad y Roles	14
6.8 Historias de Usuario	15

6.8.1 Gestión de Roles.....	15
6.8.2 Gestión de Veterinarios.....	15
6.8.3 Gestión de empleados.....	16
6.8.4 Gestión de mascotas	16
6.8.5 Gestión de turnos.....	17
6.8.6 Gestión de clientes	17
6.8.7 Gestión de razas	18
6.8.8 Gestión de especies.....	18
6.8.9 Gestión de historias clínica.....	19
6.8.10 Gestión de detalle de historias clínicas	19
6.8.11 Gestión de stock.....	20
6.8.12 Gestión de categorías.....	20
6.8.13 Gestión de productos	21
6.8.14 Gestión de ventas.....	21
6.8.15 Gestión de ventas.....	22
6.8.16 Gestión de detalles de ventas	22
6.8.17 Gestión de detalles de ventas (recepcionista)	23
6.8.18 Gestión de Informes.....	23
6.8.19 Gestión de Auditorias.....	24
6.9 Requerimientos Funcionales.....	25
6.9.1 Gestión de Roles.....	25
6.9.2 Gestión de Veterinario	25
6.9.3 Gestión de Empleados.....	25
6.9.4 Gestión de Mascotas.....	26
6.9.5 Gestión de Turnos	26
6.9.6 Gestión de Clientes	26
6.9.7 Gestión de Razas	27
6.9.8 Gestión de Especies.....	27
6.9.9 Gestión de Historia Clínica	28
6.9.10 Gestión de Detalles de Historias Clínicas	28
6.9.11 Gestión de Stock.....	28
6.9.12 Gestión de Categorías	29
6.9.13 Gestión de Productos	29
6.9.14 Gestión de Ventas	30
6.9.15 Gestión de Detalles de Ventas	30
6.9.16 Gestión de informes.....	30

6.9.17 Gestión de Auditoria	31
6.10 Requerimientos No Funcionales	31
6.10.1 Introducción	31
6.10.2 Rendimiento	31
6.10.2 Seguridad.....	32
6.10.3 Fiabilidad/Disponibilidad.....	33
6.10.4 Usabilidad.....	33
6.10.5 Mantenibilidad	34
6.11 Matriz de la Trazabilidad	35
6.12 Restricciones y Limitaciones	35
6.13 Mejoras a Futuro	35
6.14 Modelo Relacional.....	36
7. METODOLOGÍA.....	38
7.1 Enfoque Metodológico.....	38
7.2 Técnicas y Herramientas Utilizadas.....	38
7.2.1 Modelado del sistema	38
7.2.2 Tecnologías implementadas.....	38
7.3 Diseño del Sistema y Arquitectura	38
7.4 Desarrollo e Implementación	39
7.5 Base de Datos	39
7.6 Gestión de Versiones y Colaboración.....	39
7.7 Pruebas e Integración	39
7.8 Implantación y Capacitación	39
8. RECURSOS NECESARIOS	39
8.1 Recursos Humanos	39
8.2 Recursos Materiales (Hardware y Software)	40
8.3 Descripción del Equipo de Trabajo (Roles):.....	40
9. PLAN DE TRABAJO	42
9.1 Versión Descriptiva.....	42
9.2 Diagrama de Gantt	42
9.2.1 Calendarización de Actividades (Diagrama de Gantt).....	45
9.3 Etapas y entregables	46
9.4 Diagrama UML clases	46
9.5 Prototipos de interfaz	49
9.5.1 Página Principal (Dashboard o Home).....	49
9.5.2 Menú de Navegación	49

9.5.3 Modal de Gestión (Crear/Editar/Configurar)	50
9.5.4 Vista de Detalles (Ver un elemento específico)	51
9.6 Arquitectura del sistema	51
9.6.1 Patrón Arquitectónico (MVC / Cliente-Servidor)	51
9.6.2 Diagrama de Arquitectura (Nivel Alto).....	51
9.6.3 Descripción de Componentes (El Stack Tecnológico)	52
9.6.4. Flujo de Información (Cómo se conectan)	53
10 Plan de pruebas.....	53
10.1 Introducción y Objetivos	53
10.2 Alcance de las Pruebas (Scope).....	53
10.3 Entorno de Pruebas.....	54
10.4 Estrategia y Herramientas.....	54
10.5 Casos de Prueba (Muestra Representativa).....	54
10. RESULTADOS ESPERADOS	55
10.1 Resultados y Beneficios.....	55
10.2 Resultados Técnicos Alcanzados	55
10.3 Impacto Académico.....	55
10.4 Impacto Institucional y Profesional.....	55
11. REFERENCIAS.....	56

3. RESUMEN EJECUTIVO

3.1 Breve descripción del proyecto

El presente Trabajo Final Integrador consiste en el diseño y desarrollo de **SoftVet**, un Sistema de Gestión Integral para Clínicas Veterinarias, implementado como una aplicación web con arquitectura cliente–servidor.

El sistema tiene como finalidad digitalizar y optimizar los procesos administrativos y médicos de centros veterinarios, permitiendo gestionar de manera centralizada el registro de pacientes (mascotas), propietarios, turnos, historias clínicas y el control de stock de insumos. La solución incorpora una arquitectura robusta con **React** en el frontend y **Node.js** en el backend, utilizando **MySQL** para la persistencia de datos y garantizando la seguridad mediante el control de acceso basado en roles.

3.2 Objetivos principales

El objetivo general del proyecto es desarrollar una plataforma informática que permita automatizar la gestión operativa de una veterinaria, mejorando la organización de la atención médica y el control administrativo del negocio.

Como objetivos específicos se establecen:

Diseñar una base de datos relacional normalizada capaz de manejar la complejidad de las historias clínicas y turnos.

- Implementar operaciones CRUD para las entidades principales: Propietarios, Mascotas, Veterinarios y Consultas.
- Desarrollar un sistema de autenticación seguro para proteger la privacidad de los datos sensibles de los clientes.
- Optimizar la gestión de turnos para reducir tiempos de espera y organizar la agenda profesional.
- Implementar una interfaz de usuario intuitiva (UX) que facilite la carga de datos en entornos de trabajo dinámicos.
- Aplicar metodologías de desarrollo colaborativo mediante el uso de Git para la integración de módulos.

3.3 Metodología utilizada

El proyecto fue desarrollado utilizando una metodología ágil, organizada en ciclos iterativos que permitieron la validación constante de las funcionalidades.

El proceso incluyó las etapas de:

- **Relevamiento de requerimientos:** Identificación de las necesidades específicas del sector veterinario (historias clínicas, tipos de pacientes, vacunas).
- **Diseño de arquitectura:** Modelado de datos en MySQL y estructuración del backend con Express.
- **Implementación Full-stack:** Desarrollo de la lógica de negocio en el servidor y creación de componentes dinámicos en React.

- **Pruebas e Integración:** Verificación de la comunicación entre el frontend y la API, y pruebas de persistencia de datos.
- **Documentación:** Elaboración de manuales técnicos y de usuario.
- **Stack tecnológico:** Node.js, Express, MySQL, React y JavaScript moderno.

3.4 Resultados esperados

Se espera obtener un sistema funcional capaz de:

- Gestionar el ciclo de vida completo del paciente, desde el alta de la mascota hasta su historial de consultas.
- Centralizar la información de contacto de los propietarios para una comunicación más fluida.
- Sincronizar la agenda de turnos en tiempo real, evitando solapamientos.
- Garantizar la integridad y seguridad de la información clínica mediante autenticación JWT.
- Proveer una herramienta profesional que eleve el estándar de atención del centro veterinario.

3.5 Conclusiones generales

El desarrollo de **SoftVet** permitió integrar de manera práctica los conocimientos adquiridos en las áreas de desarrollo de software y análisis de datos, aplicando principios de ingeniería para resolver una problemática real del sector de salud animal.

El proyecto demuestra la capacidad de diseñar e implementar una solución tecnológica de extremo a extremo, estructurada y documentada bajo estándares académicos, evidenciando competencias en el manejo de stacks tecnológicos modernos y en la gestión de bases de datos relacionales para entornos profesionales.

4. INTRODUCCIÓN

4.1 Contexto y antecedentes

La digitalización de los servicios de salud animal se ha convertido en un requisito indispensable para la eficiencia de las clínicas veterinarias modernas. La capacidad de contar con registros clínicos centralizados y accesibles no solo optimiza la administración, sino que impacta directamente en la calidad del diagnóstico y tratamiento de los pacientes.

En el ámbito de las veterinarias de pequeña y mediana escala, aún es frecuente encontrar la gestión basada en registros físicos (fichas de papel) o herramientas ofimáticas aisladas. Esta fragmentación de la información dificulta el seguimiento histórico de las mascotas, entorpece la asignación de turnos y genera opacidad en el control de stock de medicamentos e insumos críticos.

En este escenario, **SoftVet** surge como una respuesta tecnológica integral, diseñada para unificar la gestión médica y administrativa en un entorno web seguro, permitiendo que el profesional se enfoque en la atención clínica mientras el sistema garantiza la integridad de los datos.

4.2 Justificación

La implementación de **SoftVet** responde a la necesidad de profesionalizar la gestión veterinaria mediante una plataforma que centralice el historial de los pacientes, los datos de sus propietarios y la agenda de la clínica.

Desde una perspectiva académica, este proyecto se justifica como la instancia donde se articulan y aplican conocimientos avanzados de:

- **Arquitectura de Software:** Implementación de un modelo cliente-servidor utilizando React y Node.js.
- **Persistencia de Datos:** Diseño y normalización de bases de datos relacionales en MySQL para manejar relaciones complejas (Dueño-Mascota-Consulta).
- **Seguridad:** Implementación de protocolos de autenticación y autorización para proteger la confidencialidad de las historias clínicas.
- **Análisis de Datos:** Estructuración de la información para permitir una futura toma de decisiones basada en estadísticas de atención y ventas.

4.3 Objetivo General

Desarrollar e implementar el Sistema de Gestión Integral **SoftVet**, destinado a digitalizar, centralizar y optimizar los procesos médicos y administrativos de una clínica veterinaria. El sistema busca reemplazar los métodos tradicionales de registro por una solución web moderna que facilite el seguimiento de historias clínicas, la gestión de turnos y el control de stock, mejorando la eficiencia operativa y garantizando la trazabilidad de la atención brindada a cada mascota.

4.4 Objetivos Específicos

- **Digitalizar la historia clínica:** Crear un registro estructurado y cronológico de las consultas, vacunas y tratamientos de cada paciente, facilitando el acceso inmediato a la información médica.
- **Optimizar la gestión de turnos:** Implementar un módulo de agenda que permita organizar las consultas diarias, evitando la superposición de horarios y mejorando la atención al cliente.
- **Centralizar la base de datos de clientes:** Vincular de forma eficiente a los propietarios con sus respectivas mascotas para agilizar la comunicación y el seguimiento administrativo.
- **Automatizar el control de stock de insumos:** Registrar entradas y salidas de medicamentos y productos veterinarios, garantizando que la clínica cuente siempre con los recursos necesarios para la atención.
- **Implementar seguridad por roles:** Asegurar que el personal administrativo y los médicos veterinarios accedan únicamente a las funciones y datos correspondientes a su perfil.
- **Diseñar una interfaz intuitiva y responsive:** Desarrollar un frontend en React que sea fácil de usar en el ritmo dinámico de una clínica, permitiendo la carga rápida de datos desde distintos dispositivos.

- **Garantizar la escalabilidad del sistema:** Estructurar el código y la base de datos de manera que permita la futura integración de módulos de facturación electrónica, laboratorios o recordatorios automáticos por WhatsApp.

5. ESTADO DEL ARTE

5.1 Contexto tecnológico actual

En el marco de la transformación digital, las clínicas veterinarias enfrentan el desafío de migrar de registros físicos a plataformas integradas que permitan una gestión eficiente de la salud animal y la administración del negocio. Actualmente, el mercado ofrece sistemas de Gestión de Práctica Veterinaria (VPMS) de alta complejidad, pero su adopción en clínicas de pequeña y mediana escala se ve limitada por los altos costos de suscripción y la complejidad técnica de implementación.

Los sistemas de gestión basados en la web representan una tendencia en expansión por su accesibilidad y escalabilidad. Sin embargo, muchas soluciones internacionales no se adaptan a las particularidades operativas y económicas locales. En este escenario, **SoftVet** se posiciona como una solución que cubre esta brecha, utilizando un stack tecnológico moderno (React, Node.js) que prioriza la usabilidad y la integridad de los datos médicos.

5.2 Sistemas existentes y comparativa

En el mercado actual coexisten diversas plataformas como *Cloudvet* o *ezyVet*. Estas herramientas suelen incluir:

- Gestión avanzada de historias clínicas y diagramas médicos.
- Integración con laboratorios externos.
- Facturación electrónica y reportes financieros.

No obstante, estos sistemas presentan desventajas para el sector PyME local:

- **Costos elevados:** Modelos de suscripción en moneda extranjera.
- **Rigidez:** Poca capacidad de personalización para flujos de trabajo específicos.
- **Curva de aprendizaje:** Interfaces sobrecargadas orientadas a grandes hospitales.
- **Dependencia:** Los datos residen en servidores de terceros con poca autonomía para el profesional.

5.3 Enfoques tecnológicos actuales

Predominan tres enfoques principales en la gestión veterinaria:

- **Soluciones SaaS (Software as a Service):** Como *Odoo* adaptado, ofrecen actualizaciones constantes, pero con costos recurrentes y dependencia total de internet y del proveedor.
- **Sistemas de Escritorio Tradicionales:** Ofrecen independencia de conexión, pero carecen de portabilidad y dificultan el acceso remoto desde diferentes consultorios o dispositivos móviles.
- **Aplicaciones Web con Arquitectura Modular (Caso SoftVet):** Combinan la accesibilidad multiplataforma con el control total del código. Este enfoque permite separar las responsabilidades (Frontend/Backend), facilitando el mantenimiento y la escalabilidad.

5.4 Modelado de datos y normalización

El diseño de bases de datos relacionales es el estándar para la gestión clínica debido a la necesidad de **integridad referencial**. En **SoftVet**, la normalización hasta la **Tercera Forma Normal (3FN)** es crítica para:

- Vincular de forma unívoca a cada mascota con su historial y su dueño.
- Evitar redundancias en el registro de tratamientos y vacunas.
- Garantizar la coherencia entre el stock de productos y las ventas realizadas.

5.5 Seguridad y control de acceso en aplicaciones web

La protección de datos sensibles (datos de clientes e historias clínicas) es fundamental. Se utiliza la autenticación basada en **JSON Web Token (JWT)**, consolidada como estándar para:

- Gestionar sesiones de manera segura y sin estado (stateless).
- Implementar el Control de Acceso Basado en Roles (RBAC), permitiendo que solo el veterinario edite historias clínicas mientras el personal administrativo gestiona turnos y ventas.
- Garantizar la privacidad de la información bajo principios de seguridad informática actuales.

5.6 Fundamentación del aporte del proyecto

SoftVet se diferencia por integrar los conocimientos académicos en una solución profesional:

- **Enfoque Académico-Tecnológico:** Aplicación real de un stack MERN-like (sustituyendo MongoDB por MySQL para mayor rigor relacional), con separación de responsabilidades y persistencia real.
- **Adaptación al Rubro Veterinario:** A diferencia de un ERP genérico, contempla la **Historia Clínica Digital** como eje central, el control de stock de medicamentos y la gestión de turnos médicos.
- **Código Propio y Escalable:** Permite la auditoría técnica de la calidad del código, utilizando prácticas modernas como componentes funcionales en React y middlewares de seguridad en Express.
- **Usabilidad (UX/UI):** Implementación de un diseño responsive mediante frameworks como Bootstrap o Tailwind, asegurando que el profesional pueda usar el sistema en una tablet o PC durante la consulta.

5.7 Conclusión del Estado del Arte

El análisis demuestra que existe una necesidad insatisfecha de software especializado, accesible y localizable para clínicas veterinarias pequeñas y medianas. El desarrollo de **SoftVet** permite:

- **Independencia tecnológica:** Eliminación de costos por licencias cerradas.
- **Optimización operativa:** Reducción de tiempos en la carga de historias clínicas y gestión de agenda.
- **Consolidación técnica:** Integración de los fundamentos de ingeniería de software, bases de datos y seguridad web adquiridos durante la carrera.

En conclusión, el proyecto no es solo una herramienta de gestión, sino una aplicación profesional que resuelve problemáticas reales mediante una arquitectura escalable, alineada con las exigencias del mercado tecnológico actual.

6. DESCRIPCIÓN DEL PROYECTO

6.1 Problemática Detectada

Actualmente, la gestión de la clínica veterinaria se realiza de manera analógica (papel) para historias clínicas, facturación y turnos de peluquería. Esta modalidad genera una serie de dificultades críticas:

- **Errores de Agenda:** Se pierden en promedio 8 turnos mensuales debido a confusiones en la agenda manual, lo que impacta directamente en la rentabilidad.
- **Sobrecarga Administrativa:** La carga manual de historias clínicas y ventas genera demoras en la atención y riesgos de pérdida de información médica sensible.
- **Falta de Trazabilidad:** Dificultad para rastrear el historial clínico de un paciente de forma rápida o consultar el stock de insumos y medicamentos.
- **Inconsistencia en Ventas:** La facturación a mano aumenta el margen de error humano en los cálculos y el control de caja diaria.

6.2 Solución Propuesta

Se propone el desarrollo de **SoftVet**, un Sistema de Gestión Integral implementado como aplicación web (arquitectura cliente-servidor) que busca reducir en un **70% el tiempo de gestión administrativa**.

La solución se divide en dos grandes ejes:

- **Sección Informativa/Publicitaria:** Una *Landing Page* profesional que muestra servicios, ubicación, horarios y contacto, mejorando la presencia digital del local.
- **Sección Administrativa (Panel de Gestión):** Un entorno seguro exclusivo para empleados donde se centraliza la operativa del negocio.

Módulos principales:

- **Gestión Médica:** CRUD completo de historias clínicas digitales.
- **Gestión de Agenda:** Sistema dinámico para turnos de peluquería canina y consultas.
- **Gestión Comercial:** Control de inventario, ventas de productos.
- **Seguridad:** Autenticación por roles para proteger datos de clientes y registros financieros.

6.3 Alcance del Proyecto

El sistema **SoftVet** está diseñado para cubrir el flujo completo de una clínica veterinaria y peluquería canina. **Funcionalidades incluidas:**

- Registro y administración de Clientes, Mascotas, Médicos y Personal.
- Módulo de reserva, modificación y cancelación de turnos.
- Control de Stock con actualización automática tras las ventas.

- Generación de reportes operativos y manuales técnicos/usuario.
- **Valor agregado:** Notificaciones automáticas mediante un ChatBot vía WhatsApp para recordatorios.

Limitaciones de esta versión:

- No incluye integración con pasarelas de pago (Mercado Pago, etc.); los pagos se registran como efectivo.
- No incluye facturación electrónica oficial (AFIP).
- No contempla integración con sistemas de laboratorios externos.

6.4 Análisis del Mercado

El mercado local de software veterinario en Argentina está polarizado entre sistemas internacionales costosos (*ezyVet*) y la ausencia total de tecnología en clínicas pequeñas.

SoftVet se inserta como una solución a medida para PyMEs del sector salud animal que:

- Necesitan una herramienta en español adaptada a su flujo de trabajo real.
- Buscan reducir costos operativos eliminando suscripciones en dólares.
- Requieren una interfaz clara y amigable que no exija conocimientos técnicos avanzados del personal. La propuesta se diferencia por combinar la gestión administrativa con la médica, asegurando que la digitalización no sea solo contable, sino que mejore la calidad de vida del paciente a través de su historial clínico.

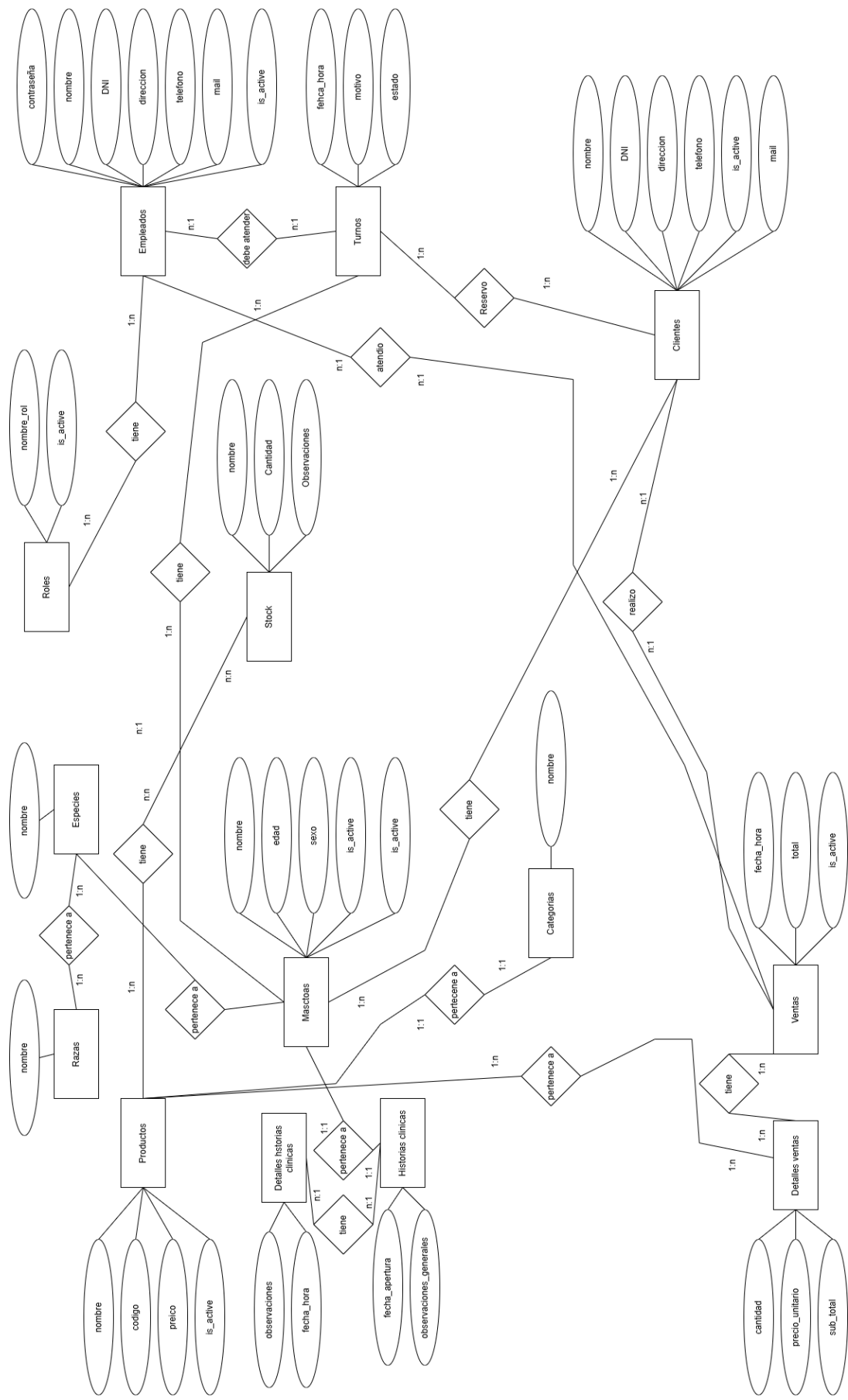
6.5 Criterios de Éxito

- **Rendimiento:** El sistema debe responder a las solicitudes en menos de 2 segundos el 98% del tiempo.
- **Eficiencia:** El usuario debe poder reservar su turno en menos de 3 minutos.
- **Compatibilidad:** Funcionamiento óptimo en navegadores Chrome, Firefox y Edge.
- **Usabilidad:** Validación mediante pruebas de usuario para garantizar una interfaz clara y amigable.

6.6 DIAGRAMA ENTIDAD-RELACIÓN

El Diagrama de Entidad-Relación (DER) es una herramienta de modelado conceptual que permite visualizar la estructura lógica de la información del sistema. Su objetivo principal es identificar las **entidades** (objetos reales o abstractos de interés para el sistema), los **atributos** que las describen y las **relaciones** que existen entre ellas.

Este diagrama facilita la comunicación entre los desarrolladores y los interesados (stakeholders) del proyecto, asegurando que los requerimientos de información estén correctamente representados antes de pasar a la implementación técnica. En este proyecto, el DER ilustra cómo interactúan los actores y los datos principales del flujo de negocio.



6.7 Reglas de Negocio

6.7.1 Gestión de Pacientes e Historias Clínicas

- **RN1. Unicidad del Paciente:** Cada mascota debe estar obligatoriamente vinculada a un Propietario (Cliente) registrado en el sistema.
- **RN2. Integridad de la Historia Clínica:** Los registros de consultas médicas o vacunas no podrán ser eliminados una vez guardados; solo podrán ser anulados mediante una observación justificada para mantener la trazabilidad médica.
- **RN3. Cronología Médica:** Las entradas en la historia clínica se ordenarán de forma cronológica descendente automáticamente por el sistema.

6.7.2 Turnos y Peluquería

- **RN4. Disponibilidad de Turnos:** No se podrá asignar un turno en un horario que ya esté ocupado por otro servicio del mismo tipo (ej. peluquería), evitando el solapamiento de agenda detectado en el diagnóstico inicial.
- **RN5. Cancelación de Turnos:** Un turno solo podrá ser cancelado o reprogramado hasta con una antelación mínima (ej. 2 horas), de lo contrario, quedará registrado como "ausente".
- **RN6. Validación de Paciente:** No se puede reservar un turno para una mascota que no esté previamente dada de alta en el sistema.

6.7.3 Productos y Stock

- **RN7. Control de Stock Crítico:** Un producto (medicamento o alimento) no podrá venderse si no existe stock suficiente, evitando saldos negativos.
- **RN8. Alerta de Reposición:** El sistema generará una alerta visual cuando el stock de un insumo alcance el mínimo configurado.
- **RN9. Persistencia de Datos:** Un producto no podrá eliminarse de la base de datos si tiene ventas; en su lugar, se marcará como "Inactivo" para preservar los reportes históricos.

6.7.4 Ventas y Cobros

- **RN10. Identificación de Operación:** Toda venta de productos o cobro de servicios (consulta/peluquería) debe estar asociada a un empleado responsable y un medio de pago válido.
- **RN11. Consistencia del Total:** El total de la transacción debe ser la suma exacta de los productos y servicios detallados, menos los descuentos aplicados.
- **RN12. Métodos de Pago:** El sistema permitirá registrar ventas únicamente en efectivo (según las limitaciones actuales del alcance), asegurando que el arqueo de caja coincida con el dinero físico.

6.7.5 Seguridad y Roles

- **RN13. Jerarquía de Acceso:** Solo los usuarios con rol "Administrador" pueden gestionar el inventario, ver reportes de facturación y administrar el personal.

- **RN14. Acceso Médico:** Solo los usuarios con rol "Veterinario" o "Administrador" tienen permisos para crear o modificar registros en las historias clínicas.
- **RN15. Sesión Única:** El acceso a los módulos administrativos requiere un token de autenticación (JWT) válido y activo.

6.8 Historias de Usuario

HU1	6.8.1 Gestión de Roles
Descripción	Como administrador del sistema, quiero gestionar los roles de usuario (crear, editar, eliminar y ver) para mantener actualizados los permisos y accesos dentro de la aplicación.
Criterios de aceptación	El administrador puede crear un nuevo rol, ingresando nombre El sistema valida que el nombre del rol no esté duplicado. El administrador puede eliminar un rol, previa confirmación mediante mensaje de advertencia. El sistema muestra un listado de todos los roles registrados (únicamente solo para administrador). Los cambios realizados se guardan correctamente en la base de datos. Solo los usuarios con rol de administrador pueden acceder a esta funcionalidad.

HU2	6.8.2 Gestión de Veterinarios
Descripción	Como administrador del sistema, quiero gestionar la información de los veterinarios (crear, editar, eliminar y visualizar) para mantener actualizada la base de datos del personal profesional.
Criterios de aceptación	El administrador puede registrar un nuevo veterinario, ingresando usuario, contraseña, nombre, DNI, dirección, teléfono, mail, id de rol. El sistema valida los campos obligatorios antes de guardar. El administrador puede editar los datos de un veterinario existente. El administrador puede eliminar un veterinario, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar un listado de todos los veterinarios registrados. Los cambios se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de administrador tienen acceso a esta funcionalidad.

--	--

HU3	6.8.3 Gestión de empleados
Descripción	Como administrador, quiero gestionar los empleados (crear, editar, eliminar, ver y asignar roles) para mantener actualizada la información del personal y controlar sus permisos dentro del sistema.
Criterios de aceptación	El administrador puede crear un nuevo empleado, ingresando usuario, contraseña, nombre, DNI, dirección, teléfono, mail, id de rol. El sistema valida los campos obligatorios antes de guardar. El administrador puede editar la información de un empleado existente, incluyendo su rol. El administrador puede eliminar un empleado, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar una lista completa de empleados registrados junto con su rol asignado. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de administrador pueden acceder a esta funcionalidad.

HU4	6.8.4 Gestión de mascotas
Descripción	Como recepcionista del sistema, quiero gestionar las mascotas (crear, editar, eliminar y visualizar) para mantener actualizada la información de los animales y su vínculo con los clientes propietarios.
Criterios de aceptación	El recepcionista puede crear una nueva mascota, ingresando nombre, edad, sexo, id de raza, id de cliente y id de historia clínica El sistema valida que se asigne una raza y un propietario antes de guardar. El recepcionista puede editar los datos de una mascota existente. El recepcionista puede eliminar una mascota, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar un listado completo de mascotas, mostrando nombre, raza y propietario asociado. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz.

	Solo los usuarios con rol de recepcionista o administrador) pueden acceder a esta funcionalidad.
--	--

HU5	6.8.5 Gestión de turnos
Descripción	Como recepcionista del sistema, quiero gestionar los turnos (crear, editar, eliminar y visualizar) para organizar las citas de los clientes evitando sobre turnos y manteniendo actualizada la información del estado de cada turno.
Criterios de aceptación	El recepcionista puede crear un nuevo turno, asignando fecha y hora, motivo, estado, id de cliente, id de mascota, id de empleado. El sistema valida que no existan sobre turnos, es decir, que no haya otro turno con el mismo veterinario, fecha y hora. El recepcionista puede editar un turno existente, modificando datos o su estado. El recepcionista puede eliminar un turno, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar una lista de turnos, mostrando cliente, mascota, veterinario, fecha, hora y estado. Los estados posibles de un turno son: <i>Pendiente, Confirmado, Cancelado y Atendido</i>. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de recepcionista o superior (por ejemplo, administrador) pueden acceder a esta funcionalidad.

HU6	6.8.6 Gestión de clientes
Descripción	Como recepcionista, quiero gestionar los clientes (crear, editar, eliminar y visualizar) para mantener actualizada la información de los propietarios y evitar registros duplicados.
Criterios de aceptación	El recepcionista puede registrar un nuevo cliente, ingresando nombre, DNI, teléfono, correo electrónico y dirección. El sistema valida que el DNI no se encuentre registrado previamente antes de guardar. El recepcionista puede editar los datos de un cliente existente. El recepcionista puede eliminar un cliente, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar un listado completo de clientes registrados.

	Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de recepcionista o superior (por ejemplo, administrador) pueden acceder a esta funcionalidad.
--	---

HU7	6.8.7 Gestión de razas
Descripción	Como recepcionista, quiero gestionar las razas (crear, editar, eliminar y visualizar) para mantener actualizada la base de datos y facilitar el registro correcto de las mascotas.
Criterios de aceptación	El recepcionista puede crear una nueva raza, ingresando nombre y especie correspondiente. El sistema valida que no existan razas duplicadas dentro de la misma especie. El recepcionista puede editar el nombre o especie de una raza existente. El recepcionista puede eliminar una raza, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar un listado de todas las razas registradas, mostrando su especie asociada. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de recepcionista o superior (por ejemplo, administrador) pueden acceder a esta funcionalidad.

HU8	6.8.8 Gestión de especies
Descripción	Como Recepcionista, quiero gestionar las especies (crear, editar, eliminar y visualizar) para mantener actualizada la base de datos y asegurar la correcta clasificación de las mascotas registradas.
Criterios de aceptación	El recepcionista puede crear una nueva especie, ingresando su nombre. El sistema valida que el nombre de la especie no esté duplicado antes de guardar. El recepcionista puede editar el nombre de una especie existente. El recepcionista puede eliminar una especie, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar un listado completo de especies registradas. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de Recepcionista o superior (por ejemplo, administrador) pueden acceder a esta funcionalidad.

--	--

HU9	6.8.9 Gestión de historias clínica
Descripción	Como veterinario o recepcionista del sistema, quiero gestionar las historias clínicas (crear, editar, eliminar y visualizar) para mantener un registro completo y actualizado de las atenciones y tratamientos de cada mascota.
Criterios de aceptación	El usuario (recepcionista o veterinario) puede crear una nueva historia clínica, asociándola a una mascota existente. La historia clínica debe incluir datos como fecha y observaciones generales El sistema valida que la historia esté asociada a una mascota registrada antes de guardar. El usuario puede editar la información de una historia clínica existente. El usuario puede eliminar una historia clínica, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar un listado de historias clínicas, mostrando mascota, fecha y veterinario responsable. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de recepcionista o veterinario pueden acceder a esta funcionalidad.

HU10	6.8.10 Gestión de detalle de historias clínicas
Descripción	Como veterinario o recepcionista del sistema, quiero gestionar los detalles de las historias clínicas (crear, editar, eliminar y visualizar) para registrar con la evolución médica de las mascotas y mantener un historial completo de sus atenciones.
Criterios de aceptación	El usuario (recepcionista o veterinario) puede crear un nuevo detalle, asociándolo a una historia clínica existente. El detalle debe incluir información como observaciones, id de historia clínica, id de venta El sistema valida que el detalle esté vinculado a una historia clínica registrada antes de guardar. El usuario puede editar la información de un detalle existente. El usuario puede eliminar un detalle, previa confirmación mediante mensaje de advertencia.

	El sistema permite visualizar un listado de detalles asociados a una historia clínica específica. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de veterinario o recepcionista pueden acceder a esta funcionalidad.
--	--

HU11	6.8.11 Gestión de stock
Descripción	Como recepcionista o administrador del sistema, quiero gestionar el stock de productos (crear, editar, eliminar y visualizar) para mantener actualizada la cantidad disponible de productos asegurar una correcta administración del inventario.
Criterios de aceptación	El usuario (recepcionista o administrador) puede registrar un nuevo movimiento de stock, seleccionando el producto correspondiente e indicando cantidad, fecha de ingreso, y observaciones. El sistema valida los campos obligatorios (producto, cantidad) antes de guardar. El usuario puede editar un registro de stock existente, modificando la cantidad, fecha de ingreso u observaciones. El usuario puede eliminar un registro de stock, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar un listado completo del stock, mostrando el nombre del producto, código, categoría, cantidad disponible, fecha de ingreso. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz de manera inmediata. El sistema actualiza automáticamente la cantidad total disponible de cada producto cuando se registren movimientos de ingreso o egreso de stock (por ejemplo, uso en tratamientos). Solo los usuarios con rol de recepcionista o administrador pueden acceder a esta funcionalidad.

HU12	6.8.12 Gestión de categorías
Descripción	Como usuario con rol de recepcionista o administrador, quiero poder ver, crear, editar y eliminar categorías de productos, para mantener una clasificación organizada que facilite la administración y búsqueda de artículos en el sistema.

Criterios de aceptación	El usuario puede registrar una nueva categoría completando los campos requeridos (nombre). El sistema valida que no exista otra categoría con el mismo nombre antes de guardarla. El usuario puede editar los datos de una categoría existente. El usuario puede eliminar una categoría previa confirmación mediante un mensaje de advertencia. El sistema muestra un listado actualizado de todas las categorías registradas. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de recepcionista_o administrador pueden acceder a esta funcionalidad.
-------------------------	---

HU13	6.8.13 Gestión de productos
Descripción	Como usuario con rol de recepcionista o administrador, quiero poder ver, crear, editar y eliminar productos, para mantener actualizado y asegurar que los productos estén correctamente clasificados y disponibles para su gestión en el sistema.
Criterios de aceptación	El usuario puede registrar un nuevo producto completando los campos requeridos (nombre, código, precio y categoría). El sistema valida que no exista otro producto con el mismo código antes de guardarlo. El usuario puede editar los datos de un producto existente. El usuario puede eliminar un producto, previa confirmación mediante mensaje de advertencia. El sistema permite visualizar un listado completo de productos, mostrando nombre, código, precio y categoría asociada. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en la interfaz. Solo los usuarios con rol de recepcionista_o administrador pueden acceder a esta funcionalidad.

HU14	6.8.14 Gestión de ventas
Descripción	Como usuario con rol de administrador, quiero poder ver, crear, editar y eliminar ventas, para mantener un registro completo de las transacciones y garantizar la correcta actualización del stock.

Criterios de aceptación	El administrador puede registrar una nueva venta completando fecha y hora, cliente, id empleado y total El sistema valida los campos obligatorios antes de guardar. El administrador puede editar una venta existente. El administrador puede eliminar una venta, previa confirmación. El sistema muestra un listado completo de ventas con toda la información relevante. Los cambios realizados se guardan correctamente en la base de datos y actualizan el stock.
-------------------------	---

HU15	6.8.15 Gestión de ventas
Descripción	Como usuario con rol de administrador, quiero poder ver, crear, editar y eliminar ventas, para mantener un registro completo de las transacciones y garantizar la correcta actualización del stock.
Criterios de aceptación	El administrador puede registrar una nueva venta completando fecha y hora, cliente, id empleado y total El sistema valida los campos obligatorios antes de guardar. El administrador puede editar una venta existente. El administrador puede eliminar una venta, previa confirmación. El sistema muestra un listado completo de ventas con toda la información relevante. Los cambios realizados se guardan correctamente en la base de datos y actualizan el stock.

HU16	6.8.16 Gestión de detalles de ventas
Descripción	Como usuario con rol de administrador, quiero poder ver, crear, editar y eliminar los detalles de cada venta, para mantener un registro completo de las transacciones y asegurar la correcta actualización del inventario.
Criterios de aceptación	El administrador puede registrar un nuevo detalle de venta asociándolo a una venta existente. Cada detalle incluye venta, producto, cantidad, precio unitario y sub total. El sistema valida que todos los campos obligatorios estén completos antes de guardar. El administrador puede editar los datos de un detalle de venta existente. El administrador puede eliminar un detalle de venta, previa confirmación. El sistema muestra un listado de todos los detalles asociados a cada venta, incluyendo producto, cantidad, precio y subtotal.

	Los cambios realizados se guardan correctamente en la base de datos y se reflejan en el stock y reportes. Solo los usuarios con rol de administrador pueden acceder a esta funcionalidad.
--	---

HU17	6.8.17 Gestión de detalles de ventas (repcionista)
Descripción	Como usuario con rol de recepcionista, quiero poder ver, crear, editar y los detalles de cada venta. No quiero poder eliminar. para mantener un registro completo de las transacciones y asegurar la correcta actualización del inventario.
Criterios de aceptación	El recepcionista puede registrar un nuevo detalle de venta asociándolo a una venta existente. Cada detalle incluye producto, cantidad, precio unitario, subtotal y observaciones. El sistema valida que todos los campos obligatorios estén completos antes de guardar. El recepcionista puede editar los datos de un detalle de venta existente. El recepcionista NO puede eliminar un detalle de venta. El sistema muestra un listado de todos los detalles asociados a cada venta, incluyendo producto, cantidad, precio y subtotal. Los cambios realizados se guardan correctamente en la base de datos y se reflejan en el stock y reportes. ☑ Solo los usuarios con rol de recepcionista pueden acceder a esta funcionalidad.

HU18	6.8.18 Gestión de Informes
Descripción	Como usuario con rol de administrador, quiero poder ver Informes de (Ventas por fechas, Turnos por fecha, Empleado con más ventas)
Criterios de aceptación	El administrador puede generar un informe con los datos recolectados. Podrá filtrar por empleado.

	Podrá filtrar por cliente. Podrá filtrar por fecha de inicio y fecha de fin. En caso de ser un turno podrá filtrar por estado (Pendiente, Realizado, Cancelado) Podrá importar estos datos en formato PDF. Solo los usuarios con rol de administrador pueden acceder a esta funcionalidad.
--	---

HU19	6.8.19 Gestión de Auditorias
Descripción	Como usuario con rol de administrador, quiero poder ver Auditorias (necesito saber quién hace cada cosa, Una Venta, Carga un Cliente, Eliminar un producto etc.)
Criterios de aceptación	El administrador puede generar un informe con los datos recolectados. Podrá filtrar por fecha de inicio y fecha de fin. Tendrá la opción de elegir para traer los datos del último día o los últimos tres días. El Informe debe mostrar Fecha y Hora, Modulo al que Pertenece, Acción Realizada, una Breve Descripción, Nombre y DNI del empleado. Podrá filtrar por Modulo, Acción, Empleado. Podrá importar estos datos en formato EXCEL. Solo los usuarios con rol de administrador pueden acceder a esta funcionalidad.

6.9 Requerimientos Funcionales

6.9.1 Gestión de Roles

Número de requisito	RF 1.0.0		
Nombre de requisito	Gestión de Roles		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir al usuario con rol de administrador crear, modificar, eliminar y visualizar los diferentes tipos de roles de usuario, con el fin de mantener actualizada la estructura de permisos del sistema.		

6.9.2 Gestión de Veterinario

Número de requisito	RF 2.0.0		
Nombre de requisito	Gestión de Veterinario		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir al usuario con rol de administrador visualizar, crear, modificar y eliminar los registros de veterinarios, con el fin de mantener actualizada la información del personal profesional.		

6.9.3 Gestión de Empleados

Número de requisito	RF 3.0.0		
Nombre de requisito	Gestión de Empleados		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir al usuario con rol de administrador visualizar, crear, modificar y eliminar los registros de empleados, así		

	como asociar un rol a cada uno de ellos, con el fin de administrar correctamente los accesos y responsabilidades dentro del sistema.
--	--

6.9.4 Gestión de Mascotas

Número de requisito	RF 4.0.0		
Nombre de requisito	Gestión de Mascotas		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Esencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir al usuario con rol de recepcionista visualizar, crear, modificar y eliminar los registros de mascotas. Cada mascota debe estar asociada a una raza y a un cliente propietario, con el objetivo de mantener organizada y actualizada la información de los animales atendidos.		

6.9.5 Gestión de Turnos

Número de requisito	RF 5.0.0		
Nombre de requisito	Gestión de Turnos		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Esencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir al usuario con rol de recepcionista visualizar, crear, modificar y eliminar los turnos registrados. Además, el sistema debe validar que no se generen sobre turnos (turnos superpuestos en fecha y hora) y permitir la asignación de un estado válido para cada turno: Pendiente, Confirmado, Cancelado o Atendido.		

6.9.6 Gestión de Clientes

Número de requisito	RF 6.0.0		
Nombre de requisito	Gestión de Clientes		

Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Esencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir al usuario con rol de recepcionista visualizar, crear, modificar y eliminar registros de clientes. Al momento de crear un nuevo cliente, el sistema debe validar que el número de DNI sea único, para evitar duplicidad en los registros.		

6.9.7 Gestión de Razas

Número de requisito	RF 7.0.0		
Nombre de requisito	Gestión de Razas		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Esencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir al usuario con rol de recepcionista visualizar, crear, modificar y eliminar los registros de razas, con el fin de mantener actualizada la información necesaria para la correcta identificación de las mascotas dentro del sistema.		

6.9.8 Gestión de Especies

Número de requisito	RF 8.0.0		
Nombre de requisito	Gestión de Especies		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Esencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir al usuario con rol de recepcionista visualizar, crear, modificar y eliminar registros de especies, con el fin de mantener actualizada la información utilizada en la clasificación de las mascotas dentro del sistema.		

6.9.9 Gestión de Historia Clínica

Número de requisito	RF 9.0.0		
Nombre de requisito	Gestión de Historia Clínica		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir a los usuarios con rol de recepcionista y veterinario visualizar, crear, modificar y eliminar registros de historias clínicas. Cada historia clínica debe estar asociada a una mascota y contener la información relevante de sus atenciones médicas, tratamientos y observaciones.		

6.9.10 Gestión de Detalles de Historias Clínicas

Número de requisito	RF 9.0.1		
Nombre de requisito	Gestión de Detalles de Historias Clínicas		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir a los usuarios con rol de recepcionista y veterinario visualizar, crear, modificar y eliminar los detalles de las historias clínicas, los cuales contienen información específica de cada atención, Estos detalles deben estar asociados a una historia clínica existente.		

6.9.11 Gestión de Stock

Número de requisito	RF 10.0.0		
Nombre de requisito	Gestión de Stock		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir a los usuarios con rol de recepcionista y administrador visualizar, crear, modificar y eliminar registros de stock		

	asociados a productos existentes. Cada registro de stock deberá incluir la cantidad disponible, fecha de ingreso, observaciones. El sistema debe mantener actualizada la cantidad total disponible de cada producto según los movimientos de stock, garantizando una gestión de inventario.
--	--

6.9.12 Gestión de Categorías

Número de requisito	RF 11.0.0		
Nombre de requisito	Gestión de Categorías		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir a los usuarios con rol de recepcionista y administrador visualizar, registrar, modificar y eliminar categorías de productos. Cada categoría deberá incluir información básica como nombre, garantizando que no existan duplicados. Esta funcionalidad permitirá organizar adecuadamente los productos dentro del sistema y facilitar su gestión.		

6.9.13 Gestión de Productos

Número de requisito	RF 12.0.0		
Nombre de requisito	Gestión de Productos		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir a los usuarios con rol de recepcionista y administrador visualizar, registrar, modificar y eliminar productos. Cada producto debe incluir información como nombre, código, precio y categoría asociada, asegurando la integridad y la correcta clasificación dentro del sistema. Esta funcionalidad permitirá mantener actualizado el catálogo de productos y facilitar su gestión en relación con el stock.		

6.9.14 Gestión de Ventas

Número de requisito	RF 13.0.0		
Nombre de requisito	Gestión de Ventas		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir: Al usuario con rol de administrador: visualizar, registrar (solo con efectivo), modificar y eliminar ventas. Al usuario con rol de recepcionista: visualizar y registrar ventas (solo con efectivo), pero no eliminar registros. Cada venta deberá incluir información como fecha y hora, cliente, id empleado y total Esta funcionalidad permitirá controlar correctamente las transacciones y mantener actualizado el stock de productos.		

6.9.15 Gestión de Detalles de Ventas

Número de requisito	RF 13.0.1		
Nombre de requisito	Gestión de Detalles de Ventas		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir: Al usuario con rol de administrador: visualizar, registrar, modificar y eliminar los detalles de cada venta, incluyendo venta, producto, cantidad, precio unitario y subtotal. Cada detalle debe estar asociado a una venta registrada, garantizando la integridad de la información y la correcta actualización del stock. Al usuario con rol de recepcionista: visualizar y registrar los detalles de ventas, pero no eliminar registros.		

6.9.16 Gestión de informes

Número de requisito	RF 14.0.0		
Nombre de requisito	Generación de informes		

Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir: Al usuario Administrador generar informes detallados a partir de los datos recolectados en la plataforma. Esta funcionalidad incluirá herramientas de filtrado múltiple para refinar las búsquedas y la capacidad de exportar los resultados a un documento para su revisión externa.		

6.9.17 Gestión de Auditoria

Número de requisito	RF 15.0.0		
Nombre de requisito	Generación de Auditoria		
Tipo	☒ Requisito	☐ Restricción	
Prioridad del requisito	☐ Alta/Eencial	☐ Media/Deseado	☐ Baja/Opcional
Descripción del requisito	El sistema debe permitir: Al usuario Administrador generar un informe de seguimiento y auditoría sobre las acciones realizadas dentro de la plataforma. La herramienta proveerá filtros específicos por módulo, usuario y fecha y permitirá descargar la información resultante en una hoja de cálculo.		

6.10 Requerimientos No Funcionales

6.10.1 Introducción

El presente documento detalla los Requerimientos No Funcionales (RNF) que definen los criterios de calidad, eficiencia, seguridad y operatividad del Sistema de Gestión de Veterinaria. Estos requerimientos son esenciales para asegurar que el sistema cumpla con las expectativas operacionales de los usuarios (Administrador, Veterinario y Recepcionista).

6.10.2 Rendimiento

ID	Requerimiento No Funcional (RNF)	Métrica / Criterio de Aceptación	Justificación (RF Asociado / Módulo)

RNF-1.1	La visualización de la ficha clínica completa de una mascota debe cargarse en un tiempo máximo.	≤ 1.5 segundos en el 95% de los casos.	Flujo de trabajo del Veterinario (historiaClinica.js).
RNF-1.2	El sistema debe soportar una cantidad mínima de usuarios realizando transacciones intensivas simultáneamente.	Mínimo de 15 usuarios concurrentes.	Operación durante horas pico (Turnos, Ventas).
RNF-1.3	La generación de reportes de ventas (RF 17.0.0) para un rango de 30 días debe tener un tiempo máximo de procesamiento.	≤ 5 segundos.	RF 17.0.0 (Gestión de Estadísticas).
RNF-1.4	La validación de sobre turnos (RF 5.0.0) debe ejecutarse de forma ágil.	≤ 200 milisegundos.	RF 5.0.0 (Gestión de Turnos) / Eficiencia de la Recepcionista.
RNF-1.5	La actualización del Stock al registrar una venta (RF 16.0.0, RF 14.0.0) debe ser una operación rápida.	≤ 1 segundo.	RF 10.0.0 / Garantizar la integridad del inventario.

6.10.2 Seguridad

ID	Requerimiento No Funcional (RNF)	Métrica / Criterio de Aceptación	Justificación (RF Asociado / Módulo)
RNF-2.1	El acceso a los módulos de gestión de empleados y roles debe ser restringido por rol.	Exclusivo para el rol de Administrador.	RF 1.0.0, RF 2.0.0, RF 3.0.0 (Gestión de Personal y Roles).
RNF-2.2	Los permisos de escritura (Crear/Modificar) sobre la Historia Clínica deben ser exclusivos.	Exclusivo para el rol de Veterinario.	RF 9.0.0, RF 9.0.1 (Integridad de la Historia Clínica).
RNF-2.3	Las comunicaciones con el servidor que manejan información sensible de clientes y empleados deben estar cifradas.	Uso de protocolo SSL/TLS.	RF 6.0.0, RF 3.0.0 / Protección de datos personales.
RNF-2.4	El sistema debe utilizar un mecanismo de almacenamiento seguro para las credenciales de los empleados.	Uso de un algoritmo de <i>hashing</i> robusto (Ej: BCrypt o Argon2) para contraseñas.	RF 3.0.0 (Gestión de Empleados).

RNF-2.5	El sistema debe registrar cualquier cambio realizado en los registros médicos.	Implementación de un log de auditoría inmutable para modificaciones en Historia Clínica.	RF 9.0.0, RF 9.0.1 (Auditoría Médica).
RNF-2.6	El sistema debe exigir el cumplimiento de una política de contraseñas.	Mínimo 8 caracteres, mayúsculas, minúsculas, números y símbolos.	Seguridad general de acceso.

6.10.3 Fiabilidad/Disponibilidad

ID	Requerimiento No Funcional (RNF)	Métrica / Criterio de Aceptación	Justificación (RF Asociado / Módulo)
RNF-3.1	El sistema debe mantener su operatividad durante el horario laboral.	Disponibilidad mínima del 99.5% (24/7 o en horario de atención).	Operación crítica de la clínica.
RNF-3.2	Se debe implementar un procedimiento de resguardo de datos.	Copia de seguridad completa de la base de datos diariamente.	Recuperación ante fallos.
RNF-3.3	El sistema debe recuperarse completamente ante una falla total (RTO).	Recuperación total en menos de 4 horas.	Continuidad del negocio (Turnos, Ventas).
RNF-3.4	El subsistema de Gestión de Turnos y Clientes (RF 5.0.0, RF 6.0.0) debe tener la máxima prioridad de restauración.	Prioridad 1 en el Plan de Recuperación de Desastres.	Flujos de trabajo críticos de la Recepcionista.

6.10.4 Usabilidad

2	Requerimiento No Funcional (RNF)	Métrica / Criterio de Aceptación	Justificación (RF Asociado / Módulo)
RNF-4.1	La interfaz de gestión de Turnos (RF 5.0.0) debe minimizar los pasos para programar una cita.	No requerir más de 4 pasos para el registro de un nuevo turno.	RF 5.0.0 / Optimización de la Recepcionista.

RNF-4.2	El diseño de la interfaz debe ser adaptable.	Interfaz responsive y compatible con tabletas.	Uso del Veterinario en áreas de consulta.
RNF-4.3	Se debe proporcionar retroalimentación clara al usuario ante errores de entrada.	Mensajes de error claros que sugieran la acción correctiva.	Experiencia de usuario (Clientes, Mascotas).
RNF-4.4	El proceso inicial de creación de un nuevo Cliente (RF 6.0.0) debe ser rápido.	Máximo de 3 campos obligatorios en la primera pantalla de registro.	RF 6.0.0 / Agilizar el <i>check-in</i> de nuevos clientes.
RNF-4.5	La interfaz de Registro de Ventas (RF 16.0.0) debe incorporar una ayuda visual para encontrar productos.	Inclusión de función de búsqueda predictiva de productos.	RF 16.0.0 / Acelerar la transacción de venta.

6.10.5 Mantenibilidad

ID	Requerimiento No Funcional (RNF)	Métrica / Criterio de Aceptación	Justificación (RF Asociado / Módulo)
RNF-5.1	El código fuente debe adherirse a un conjunto de reglas para garantizar su uniformidad y legibilidad.	Uso obligatorio de un estándar de codificación (Ej: ESLint o Prettier).	Facilitar la incorporación de nuevos desarrolladores.
RNF-5.2	La arquitectura del sistema debe ser modular.	La modificación en un módulo (Ej: ventas.js) no debe impactar la funcionalidad de otros módulos principales (Ej: ventas.js).	Baja cohesión/Alto acoplamiento.
RNF-5.3	El sistema debe mantenerse actualizado con las tecnologías base.	Compatibilidad con la última versión estable de Node.js y el motor de base de datos utilizado.	Portabilidad y seguridad futura.
RNF-5.4	La lógica de cálculo de estadísticas debe ser independiente de la lógica transaccional.	Módulo de Estadísticas (RF 17.0.0) desacoplado de Ventas y Stock.	Permitir modificaciones de reportes sin afectar la operación diaria.

6.11 Matriz de la Trazabilidad

HISTORIA DE USUARIO	DESCRIPCIÓN BREVE	REGLA DE NEGOCIO	REQUISITO FUNCIONAL
HU01	Registrar Historia Clínica	RN1 - RN2 - RN3	RF01 - Gestión Médica
HU02	Agendar Turno Peluquería	RN4 - RN5 - RN6	RF02 - Gestión Agenda
HU03	Registrar Venta / Stock	RN7 - RN8 - RN9	RF03 - Gestión Inventario
HU04	Realizar Cobro / Caja	RN10 - RN11 - RN12	RF04 - Gestión Caja
HU05	Administrar Personal	RN13 - RN14 - RN15	RF05 - Gestión Usuarios

6.12 Restricciones y Limitaciones

- **Tecnología del Cliente:** Navegadores web modernos (Chrome, Firefox, Edge) con soporte para JavaScript (ES6+).
- **Base de Datos:** Motor relacional **MySQL** versión 8.0 o superior, configurado para persistencia local.
- **Entorno de Ejecución:** Servidor local basado en **Node.js**.
- **Seguridad de Acceso:** Acceso restringido mediante autenticación **JWT** (JSON Web Token) y una sesión activa por usuario.
- **Métodos de Pago:** En esta instancia, el sistema solo procesa y registra transacciones en **efectivo**.
- **Infraestructura:** El despliegue inicial se realiza en entorno local (localhost), sin hosting en la nube para esta etapa académica.

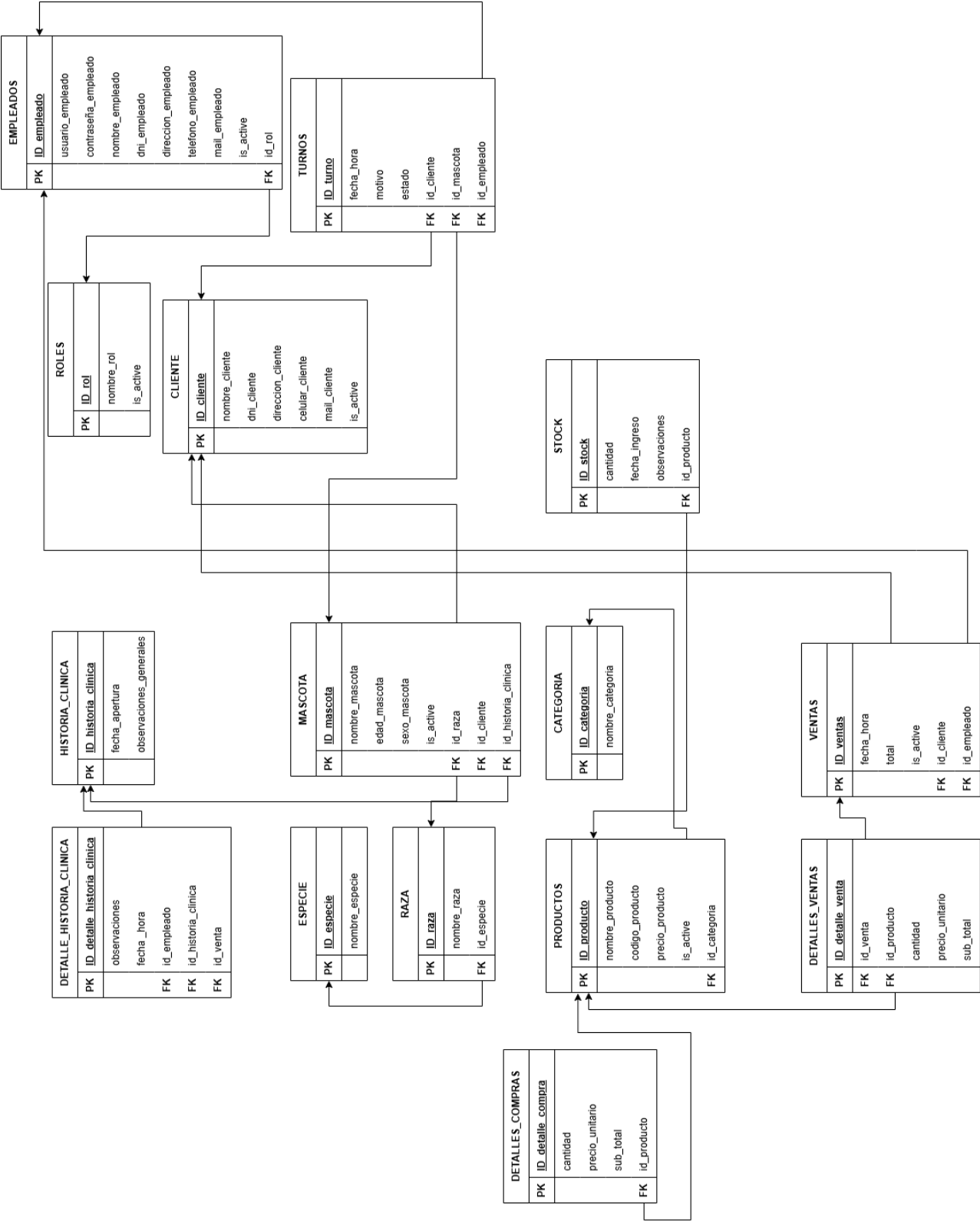
6.13 Mejoras a Futuro

- **Notificaciones Inteligentes:** Implementar el envío de recordatorios de turnos y alertas de vacunación automáticamente por el **ChatBot de WhatsApp**.
- **Facturación Electrónica:** Integración con la API de AFIP para la emisión de facturas legales.
- **Dashboard de Análisis de Datos:** Incorporar métricas avanzadas y gráficos estadísticos sobre el crecimiento de pacientes y rentabilidad mensual.

- **Módulo de Laboratorio:** Permitir la carga de archivos (PDF/Imágenes) con resultados de análisis clínicos directamente en la historia clínica.
- **Integración de Pagos:** Conexión con pasarelas de pago digitales (Mercado Pago / tarjetas) para cobros en línea de señas de turnos.
- **Versión Mobile:** Desarrollo de una aplicación móvil nativa para que los dueños de mascotas puedan consultar el historial de vacunas de sus animales.

6.14 Modelo Relacional

Representa la estructura técnica de la base de datos tal como será implementada en el Sistema de Gestión de Bases de Datos (SGBD). En esta etapa, el modelo conceptual se transforma en un conjunto de **tablas** normalizadas. Este diagrama detalla las columnas (campos), los tipos de datos, las **claves primarias (PK)** que identifican registros únicos y las **claves foráneas (FK)** que establecen la integridad referencial entre las tablas.



7. METODOLOGÍA

7.1 Enfoque Metodológico

Para el desarrollo de **SoftVet** se adoptó una metodología ágil basada en **Scrum**, organizada en ciclos iterativos denominados *sprints*. Este enfoque permitió una construcción incremental del sistema, facilitando la detección temprana de errores en la lógica médica (historias clínicas) y administrativa.

El desarrollo se estructuró en las siguientes fases:

- **Relevamiento:** Identificación de falencias en la gestión manual (pérdida de turnos).
- **Modelado:** Diseño del DER (Entidad-Relación) para pacientes, médicos y turnos.
- **Implementación:** Desarrollo del stack Full-stack (Frontend y Backend).
- **Pruebas:** Validación de reglas de negocio (ej. no duplicar turnos en peluquería).
- **Documentación:** Elaboración de manuales técnicos y de usuario finales.

7.2 Técnicas y Herramientas Utilizadas

7.2.1 Modelado del sistema

Se utilizó el modelado entidad-relación para MySQL, garantizando:

- **Normalización (3FN):** Para evitar la redundancia entre dueños y múltiples mascotas.
- **Integridad Referencial:** Claves foráneas para vincular consultas con médicos y pacientes.

7.2.2 Tecnologías implementadas

- **Backend:** Node.js con Express.js. Se utilizaron middlewares como **CORS** para comunicación entre puertos, **bcryptjs** para el cifrado de claves y **jsonwebtoken** (JWT) para la seguridad de sesiones.
- **Frontend:** **React.js** como librería principal, permitiendo una interfaz dinámica basada en componentes. Para el estilado se utilizaron **TailwindCSS** y **Bootstrap**, logrando un diseño *responsive* y profesional.
- **Base de Datos:** MySQL como motor relacional robusto.
- **Herramientas:** Postman para el testeo de la API Rest y Git para el control de versiones.

7.3 Diseño del Sistema y Arquitectura

Se implementó una arquitectura de **tres capas**:

1. **Capa de Presentación (Frontend):** Desarrollada en React, organizada por componentes reutilizables (formularios de alta, tablas de turnos).
2. **Capa de Lógica (Backend):** Una API REST estructurada en Controladores, Rutas y Middlewares para separar responsabilidades.
3. **Capa de Datos:** Base de datos MySQL con persistencia real.

El diseño visual se trabajó priorizando la **UX (Experiencia de Usuario)** clínica, utilizando menús laterales para un acceso rápido a la Historia Clínica y la Agenda de Turnos durante la atención veterinaria.

7.4 Desarrollo e Implementación

El desarrollo fue **colaborativo**, gestionando ramas de trabajo para integrar módulos de manera eficiente.

- **Frontend:** Se crearon paneles específicos para la gestión de mascotas, visualización de historias clínicas y un módulo de reserva de turnos de peluquería con validación de tiempo real.
- **Backend:** Se desarrollaron endpoints seguros. La lógica se centró en asegurar que cada registro médico esté inalterablemente vinculado a un profesional y un paciente.

7.5 Base de Datos

MySQL permitió establecer una estructura sólida. Se implementaron **Triggers y Constraints** para:

- Evitar la venta de productos sin stock.
- Impedir el borrado accidental de historias clínicas con registros asociados.
- Mantener la consistencia en la relación Propietario-Mascota.

7.6 Gestión de Versiones y Colaboración

Se utilizó **GitHub** como repositorio central. El flujo de trabajo incluyó el *merge* de ramas funcionales (como las desarrolladas en colaboración con Davidvg29), asegurando que la rama principal (main) siempre contuviera una versión estable del sistema. El entorno de desarrollo principal fue Visual Studio Code.

7.7 Pruebas e Integración

Se realizaron pruebas de integración para asegurar que:

- Los formularios de React validaran correctamente los datos antes de enviarlos a Node.js.
- La seguridad JWT denegara el acceso a módulos administrativos a usuarios no autorizados.

7.8 Implantación y Capacitación

El sistema se desplegó en un entorno local de pruebas. Se elaboró un **Manual de Usuario** enfocado en el personal de la clínica, detallando desde el alta de un nuevo paciente hasta la generación de reportes de ventas y cierres de caja. Se incluyó un canal de soporte para la resolución de dudas técnicas durante la fase de implementación inicial.

8. RECURSOS NECESARIOS

8.1 Recursos Humanos

Para el ciclo de vida de **SoftVet**, se definieron roles específicos que cubren tanto la parte técnica como la analítica y legal (necesaria para el manejo de datos sensibles):

- **Analista Funcional y de Datos:** Encargado del relevamiento de requerimientos, definición de reglas de negocio y estructuración de la información para futuros reportes estadísticos.

- **Desarrollador Full-Stack (React & Node.js):** Responsable de la implementación de la lógica de negocio en el backend y de la creación de la interfaz de usuario dinámica en el frontend.
- **Diseñador de Base de Datos:** Encargado del modelado entidad-relación en MySQL, garantizando la integridad de las historias clínicas y la normalización de tablas.
- **Especialista en Seguridad y Documentación:** Responsable de implementar la autenticación por roles (JWT), asegurar el cumplimiento de la privacidad de datos y redactar los manuales técnicos y de usuario.
- **Colaborador Externo (DevOps/Git):** Apoyo en la integración de ramas de código y control de versiones a través de GitHub.

8.2 Recursos Materiales (Hardware y Software)

Hardware:

- Equipos de cómputo personales con procesadores de arquitectura x64.
- Dispositivos móviles (Android/iOS) para pruebas de la Landing Page responsiva.

Software y Entorno de Desarrollo:

- Sistema Operativo: Windows 10/11.
- IDE: Visual Studio Code con extensiones para desarrollo en JavaScript y React.
- Entorno de Ejecución: Node.js (Runtime) y gestor de paquetes NPM.
- Gestor de Base de Datos: MySQL Workbench y servidor local para la persistencia de datos.
- Control de Versiones: Git y cliente de escritorio para la gestión de repositorios en GitHub.
- Herramientas de Testing: Postman para la validación de la API Rest.

Recursos de Terceros (Librerías):

- Frameworks de estilos: **TailwindCSS** y **Bootstrap**.
- Seguridad: **Bcryptjs** (encriptación) y **JsonWebToken** (autenticación).
- Utilidades: Librerías para generación de reportes en PDF/Excel

8.3 Descripción del Equipo de Trabajo (Roles):

INTEGRANTE	ROL	DESCRIPCION
Celiz Mariano	Analista Funcional Desarrollador Frontend Desarrollador Backend Diseñador de Base de Datos Responsable de Documentación	Encargado del diseño visual del sistema en React, estructura de pantallas responsivas, validaciones de formularios y documentación técnica. Responsable del desarrollo de la lógica del servidor en Node.js, autenticación con JWT, controladores, conexión con MySQL y seguridad del sistema. Diseño y administró la base de datos relacional, gestionando tablas, relaciones

		(Dueño-Mascota-Historia Clínica), integridad de datos y consultas SQL optimizadas.
Diaz Joaquin	Analista Funcional Desarrollador Frontend Desarrollador Backend Diseñador de Base de Datos Responsable de Documentación	Encargado del diseño visual del sistema en React, estructura de pantallas responsivas, validaciones de formularios y documentación técnica. Responsable del desarrollo de la lógica del servidor en Node.js, autenticación con JWT, controladores, conexión con MySQL y seguridad del sistema. Diseño y administró la base de datos relacional, gestionando tablas, relaciones (Dueño-Mascota-Historia Clínica), integridad de datos y consultas SQL optimizadas.

INTEGRANTE	ROL	DESCRIPCION
Pastrana Angel	Analista Funcional Desarrollador Frontend Desarrollador Backend Diseñador de Base de Datos Responsable de Documentación	Encargado del diseño visual del sistema en React, estructura de pantallas responsivas, validaciones de formularios y documentación técnica. Responsable del desarrollo de la lógica del servidor en Node.js, autenticación con JWT, controladores, conexión con MySQL y seguridad del sistema. Diseño y administró la base de datos relacional, gestionando tablas, relaciones (Dueño-Mascota-Historia Clínica), integridad de datos y consultas SQL optimizadas.
Valdez Gramajo David	Analista Funcional Desarrollador Frontend Desarrollador Backend Diseñador de Base de Datos Responsable de Documentación	Encargado del diseño visual del sistema en React, estructura de pantallas responsivas, validaciones de formularios y documentación técnica. Responsable del desarrollo de la lógica del servidor en Node.js, autenticación con JWT, controladores, conexión con MySQL y seguridad del sistema. Diseño y administró la base de datos relacional, gestionando tablas, relaciones (Dueño-Mascota-Historia Clínica), integridad de datos y consultas SQL optimizadas.

9. PLAN DE TRABAJO

9.1 Versión Descriptiva

El desarrollo del Trabajo Final Integrador se organizó en etapas progresivas, estructuradas bajo un enfoque iterativo basado en *sprints*, permitiendo dividir el proyecto en bloques funcionales y facilitar el control del avance.

9.2 Diagrama de Gantt

El proyecto **SoftVet** fue planificado mediante un cronograma de trabajo que abarca desde mediados de septiembre de 2025 hasta las instancias finales de entrega. La ejecución se estructuró en tres grandes etapas, diferenciadas visualmente en el diagrama.

1. Fase Inicial y Diseño (Barras Lila Claro)

- Esta fase comprende las tareas preliminares ejecutadas a partir del 15 de septiembre de 2025, fundamentales para sentar las bases del sistema:
- Relevamiento inicial: análisis de requerimientos y reglas de negocio.
- Diseño de base de datos: modelado de entidades y relaciones.
- Creación de landing page: desarrollo de la interfaz inicial del sistema.

Duración de la fase: desde el 15/09/2025 hasta el 30/09/2025.

2. Fase de Desarrollo Iterativo – Sprints 1 al 7 (Barras Violeta Intenso)

El núcleo del desarrollo se organizó en **7 sprints consecutivos**, de aproximadamente dos semanas de duración cada uno. En esta etapa se implementaron de forma incremental las distintas funcionalidades definidas en las historias de usuario, abarcando la gestión integral del sistema.

- **Sprint 1 (Inicio de octubre):** Desarrollo de la gestión de roles, veterinarios y empleados, junto con la administración de mascotas y turnos.
Duración: del 01/10/2025 al 15/10/2025.
- **Sprint 2 (Mediados de octubre):** Implementación del módulo médico (historias clínicas y sus detalles), junto con la gestión de clientes.
Duración: del 16/10/2025 al 31/10/2025.
- **Sprint 3 (Inicio de noviembre):** Desarrollo de la gestión de productos, categorías y stock.
Duración: del 01/11/2025 al 15/11/2025.
- **Sprint 4 (Mediados de noviembre):** Implementación del módulo de razas y especies, incluyendo sus respectivos detalles.
Duración: del 17/11/2025 al 01/12/2025.
- **Sprint 5 (Inicio de diciembre):** Desarrollo del módulo de ventas, incluyendo gestión de ventas y sus detalles, tanto para administrador como recepcionista.
Duración: del 01/12/2025 al 15/12/2025.
- **Sprint 6 (Mediados de diciembre):** Revisión general del sistema, ajustes en la base de datos y mejoras en el frontend, incluyendo login y vistas diferenciadas según el rol del

usuario.

Duración: del 15/12/2025 al 30/12/2025.

- **Sprint 7 (Febrero):** Implementación del módulo de auditorías e informes, junto con la incorporación de filtros de búsqueda por fecha, estado y otros criterios en distintos módulos del sistema.

Duración: del 23/02/2026 al 09/03/2026.

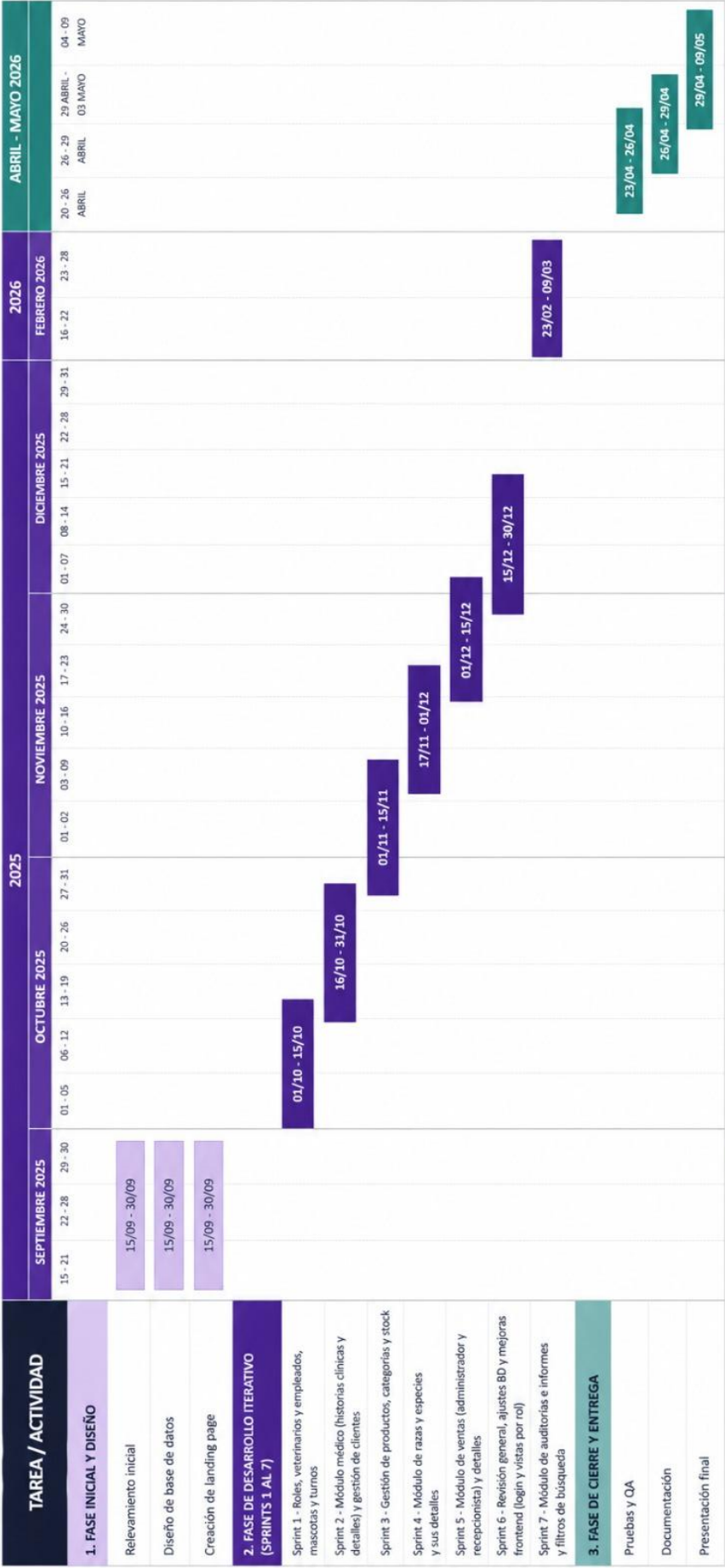
3. Fase de Cierre y Entrega (Barras Verde Agua)

La etapa final se dedicó a asegurar la calidad del sistema y formalizar la entrega del proyecto:

- **Pruebas y QA:** verificación de funcionalidades y corrección de errores.
Duración: del 23/04/2026 al 26/04/2026.
- **Documentación:** redacción de manual técnico y manual de usuario.
Duración: del 26/04/2026 al 29/04/2026.
- **Presentación final:** entrega y defensa del Trabajo Final Integrador.
Duración: del 29/04/2026 al 09/05/2026.

Duración total de la fase: desde el 23/04/2026 hasta el 09/05/2026.

Diagrama de Gantt - Proyecto SoftVet



9.2.1 Calendarización de Actividades (Diagrama de Gantt)

ACTIVIDAD	FECHA DE INICIO	FECHA DE FIN	DURACIÓN	RESPONSABLE
RELEVAMIENTO Y ANÁLISIS	15/09/2025	17/09/2025	3 días	Mariano
DISEÑO DE BASE DE DATOS	18/09/2025	24/09/2025	5 días	David
CREACIÓN DE LANDING PAGE	25/09/2025	01/10/2025	5 días	Joaquín
SPRINT 1: Gestión de roles, veterinarios, empleados, mascotas y turnos.	01/10/2025	15/10/2025	11 días	Ángel
SPRINT 2: Módulo médico (historias clínicas) y clientes.	16/10/2025	31/10/2025	12 días	Mariano
SPRINT 3: Gestión de productos, categorías y stock.	01/11/2025	15/11/2025	10 días	Joaquín
SPRINT 4: Módulo de razas y especies.	17/11/2025	01/12/2025	11 días	Ángel
SPRINT 5: Módulo de ventas (admin y recepcionista).	01/12/2025	15/12/2025	11 días	Mariano y Joaquín
SPRINT 6: Revisión general, ajustes de BD, login y vistas por rol.	15/12/2025	30/12/2025	12 días	David
SPRINT 7: Auditorías, informes y filtros de búsqueda.	23/02/2026	09/03/2026	11 días	David y Ángel
DOCUMENTACIÓN Y CARPETA TFI	10/03/2026	24/03/2026	11 días	Grupo completo

9.3 Etapas y entregables

ETAPAS	ACTIVIDADES PRINCIPALES	ENTREGABLE
1. Análisis	Relevamiento de procesos veterinarios. Definición de reglas de negocio y HU.	Documento de requerimientos
2. Diseño	Modelado ER en MySQL. Diseño de arquitectura y Mockups.	Diagrama ER y estructura del sistema
3. Backend	CRUD de pacientes, historias clínicas. Autenticación JWT y lógica de negocio.	API funcional en Node.js
4. Frontend	Desarrollo de interfaces en React. Integración con API y diseño responsive.	Sistema con interfaz completa
5. Pruebas	Testing de turnos y validación de stock. Corrección de errores y QA.	Sistema estable y funcional
6. Documentación	Manual técnico y de usuario. Revisión final de la carpeta TFI.	Carpeta final lista

9.4 Diagrama UML clases

La arquitectura se organiza en tres módulos lógicos que interactúan entre sí:

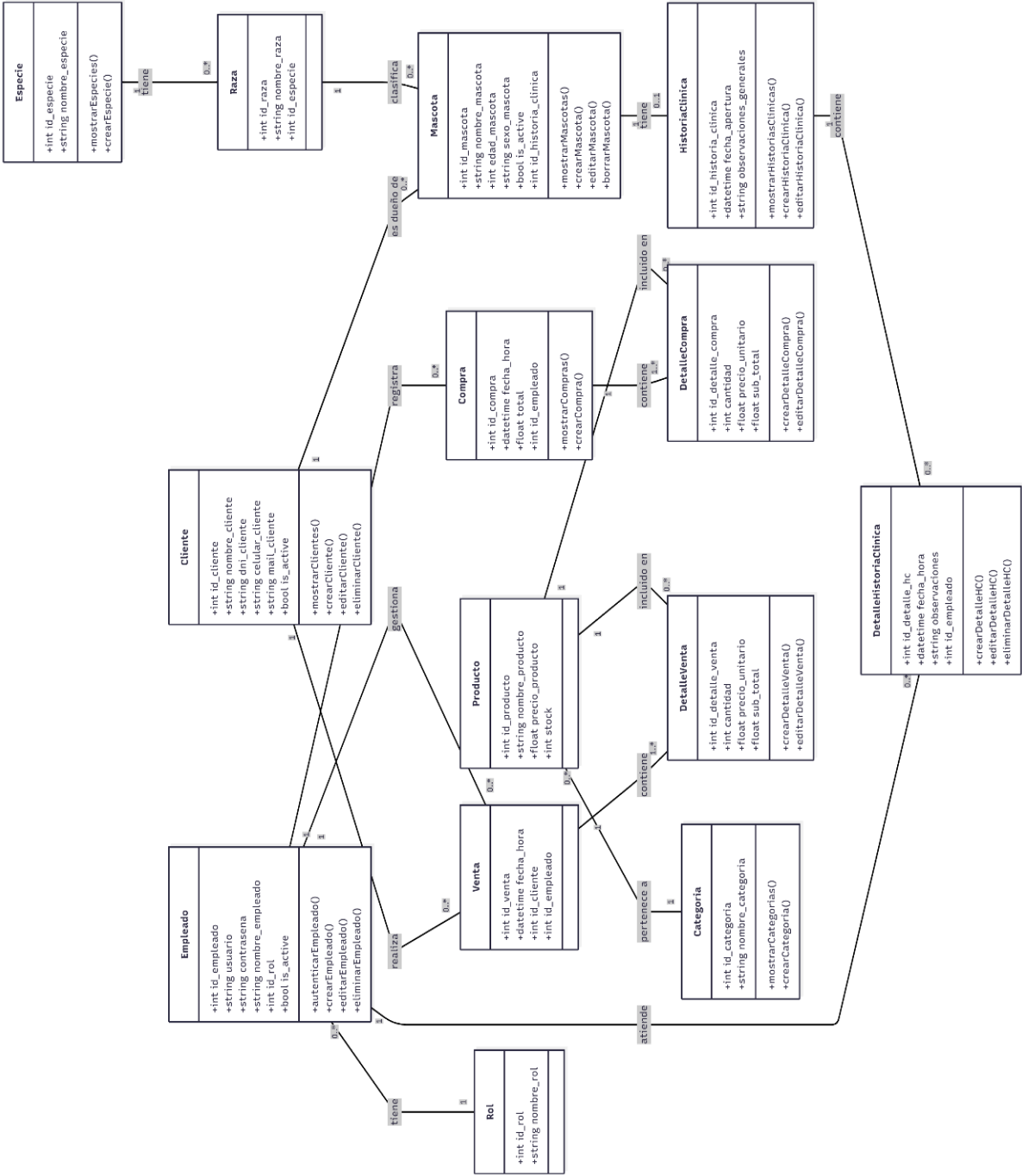
- 1. Módulo Administrativo y de Seguridad** Este módulo gestiona los actores del sistema. La clase central es Empleado, la cual contiene las credenciales de acceso (usuario, contraseña) y determina los permisos a través de su relación con la clase Rol (relación 1 a muchos). Por otro lado, la clase Cliente representa a los usuarios externos (dueños de mascotas). Ambas clases poseen métodos para la gestión de sus perfiles (crear, editar, eliminar) y atributos de estado (is_active) para el borrado lógico.
- 2. Módulo Veterinario (Pacientes y Clínica)** Modelado para garantizar la integridad de los datos médicos:

- **Jerarquía de Clasificación:** Se establece una cadena de dependencia donde una Especie agrupa múltiples Razas, y una Raza clasifica a una Mascota.
- **Historia Clínica:** Cada Mascota posee una única Historia Clínica. Sin embargo, para registrar la evolución del paciente a lo largo del tiempo, la historia clínica se compone de múltiples registros de Detalle Historia Clínica.
- **Trazabilidad:** Es importante destacar que cada Detalle Historia Clínica está vinculado a un Empleado específico, lo que permite auditar qué profesional realizó cada atención médica.

3. Módulo Comercial (Inventario y Transacciones) Este módulo administra el flujo económico y de stock:

- **Catálogo:** Los productos (Producto) se organizan mediante la clase Categoría y gestionan el inventario a través del atributo stock.
- **Transacciones (Ventas):** Se implementa un patrón maestro-detalle tanto para las ventas (Venta y Detalle Venta).
- Las clases "padre" (Venta) almacenan los datos de cabecera (fecha, total, empleado responsable).
- Las clases "detalle" almacenan los ítems específicos, cantidades y subtotales, vinculándose directamente con el Producto involucrado.

Relaciones Clave: El diagrama destaca la centralidad del Empleado como el operador principal del sistema, teniendo participación directa en la gestión de ventas (gestiona) y la atención médica de los pacientes (atiende).



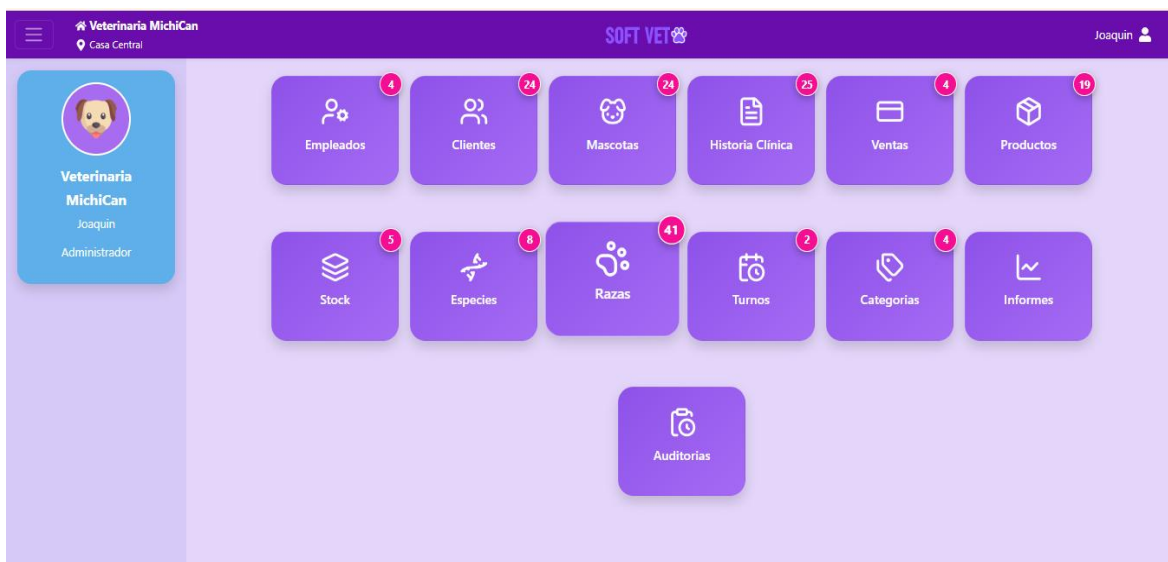
9.5 Prototipos de interfaz

En esta sección se presentan los prototipos de interfaz de usuario (UI) diseñados **inicialmente** para el sistema. Estos mockups representan la visualización conceptual de las pantallas principales y sirvieron como **referencia base** para la maquetación del frontend.

El objetivo principal de estos prototipos fue validar la experiencia de usuario (UX), proponer una disposición de los elementos y establecer un flujo de navegación lógico. Cabe destacar que **estas vistas no constituyen el diseño final estático**. Al tratarse de una maquetación preliminar basada en referencias visuales, el diseño **evolucionó durante la etapa de desarrollo**. Se realizaron modificaciones y ajustes sobre la marcha para optimizar la usabilidad, resolver desafíos técnicos emergentes y adaptar la interfaz a las necesidades funcionales reales que surgieron durante la implementación.

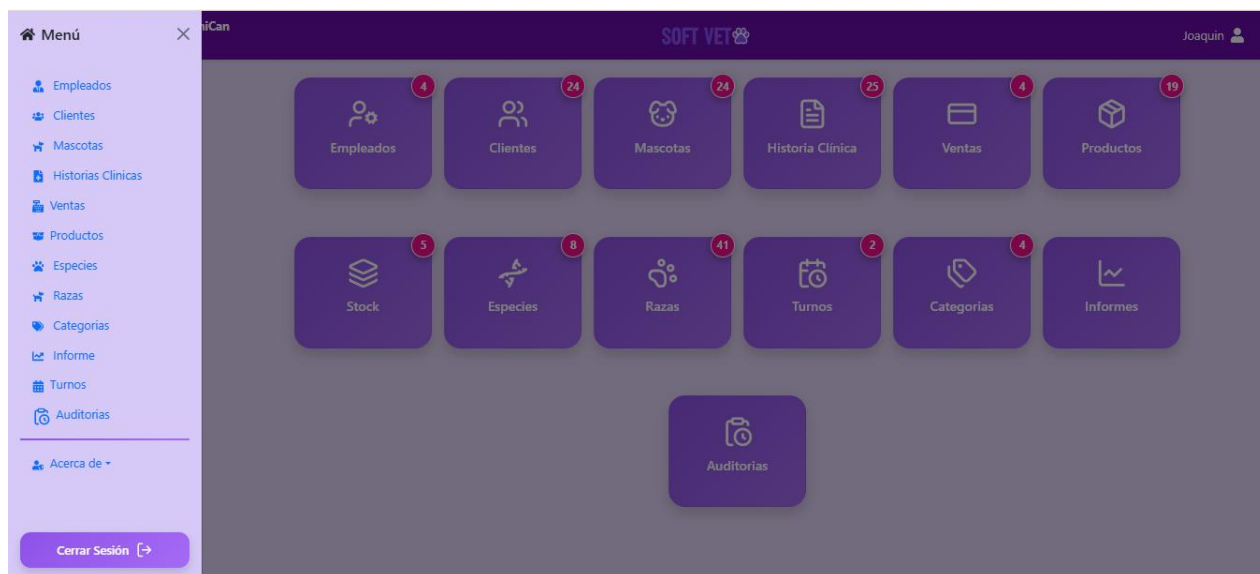
9.5.1 Página Principal (Dashboard o Home)

Figura 1. Vista de la Página Principal (Dashboard) "Esta pantalla constituye el punto de entrada centralizado al sistema tras la autenticación. Su propósito es ofrecer al usuario una visión panorámica del estado actual de la aplicación, accesos directos a las funcionalidades más utilizadas y un resumen de la actividad reciente, facilitando así la toma de decisiones rápidas y la navegación hacia módulos específicos.



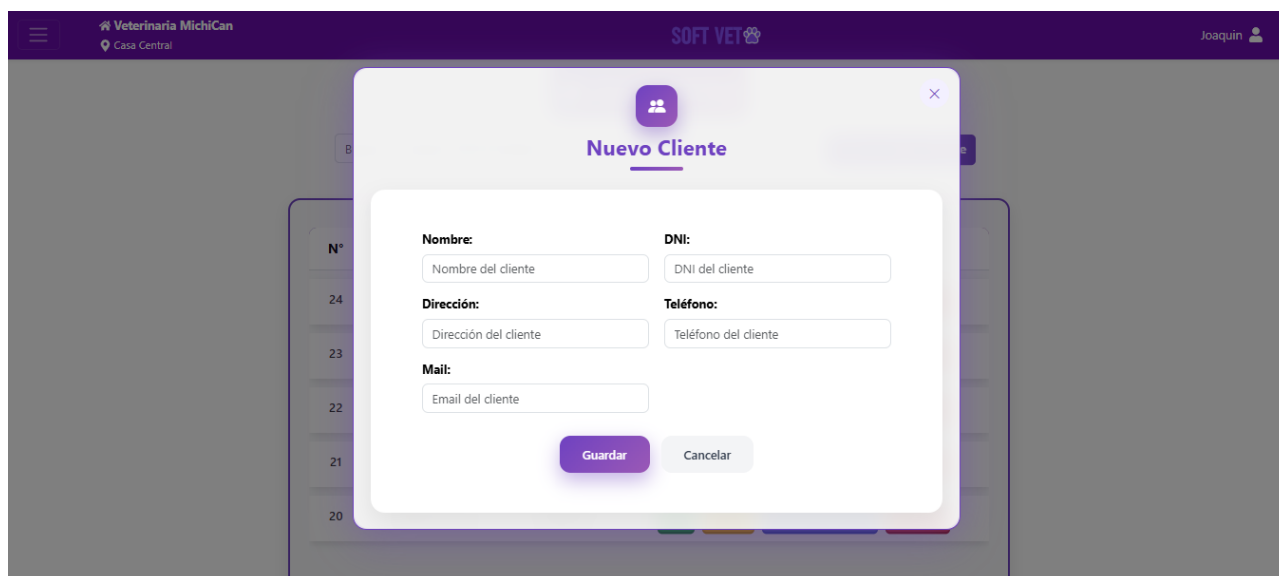
9.5.2 Menú de Navegación

Figura 2. Estructura del Menú de Navegación Principal "Se ilustra el componente de navegación primario del sistema [puedes añadir aquí: (barra lateral / menú superior)]. Este elemento define la arquitectura de la información y la jerarquía de los módulos disponibles. Su diseño prioriza la claridad y la accesibilidad, permitiendo al usuario orientarse dentro de la aplicación y desplazarse ágilmente entre las distintas secciones funcionales.



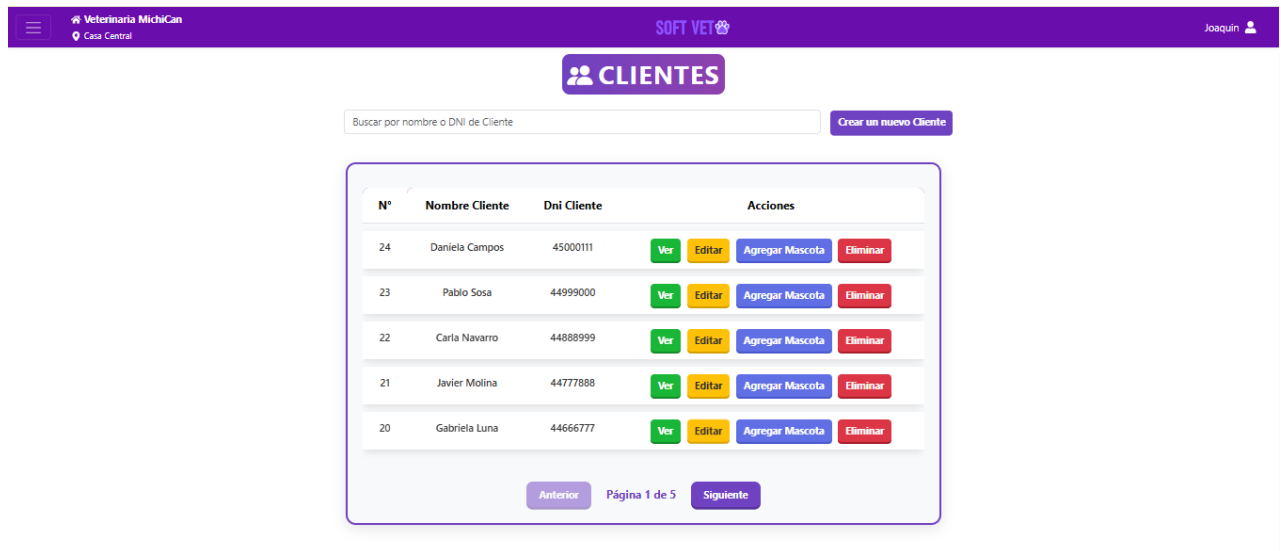
9.5.3 Modal de Gestión (Crear/Editar/Configurar)

Figura 3. Ventana Modal para Gestión de Registros "La imagen muestra la implementación de una ventana modal (pop-up) diseñada para realizar tareas de gestión puntuales, como la creación de un nuevo registro. Este patrón de interfaz permite al usuario completar una acción transaccional sin perder el contexto visual de la pantalla subyacente, agilizando el flujo de trabajo y manteniendo el foco en la tarea actual.



9.5.4 Vista de Detalles (Ver un elemento específico)

Figura 4. Vista Detallada de Entidad "Esta interfaz se despliega al seleccionar un elemento específico desde un listado general. El objetivo de la vista de detalles es presentar de forma exhaustiva y estructurada toda la información granular asociada a dicha entidad (atributos, historial, datos relacionados), permitiendo una revisión profunda y focalizada del registro seleccionado.



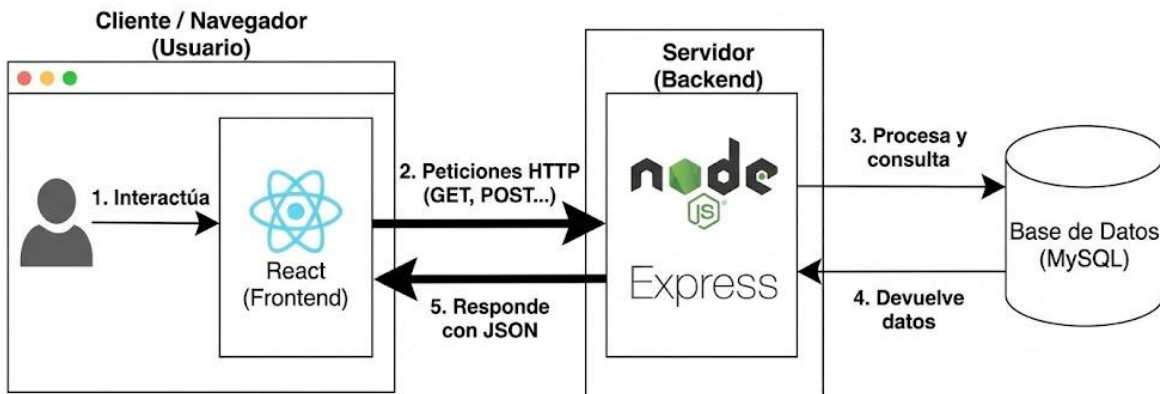
9.6 Arquitectura del sistema

9.6.1 Patrón Arquitectónico (MVC / Cliente-Servidor)

El sistema se ha desarrollado bajo una arquitectura de **Aplicación de Página Única (SPA)** siguiendo el modelo **Cliente-Servidor**. Esta estructura desacopla completamente la lógica de presentación (Frontend) de la lógica de negocio y acceso a datos (Backend), permitiendo que ambas partes evolucionen de manera independiente."

9.6.2 Diagrama de Arquitectura (Nivel Alto)

- **Navegador (Usuario)** interactúa con React.
- **React** envía peticiones HTTP (GET, POST...) a Node/Express.
- **Express** procesa y consulta a MySQL.
- **MySQL** devuelve datos a Express.
- **Express** responde con JSON a React.



9.6.3 Descripción de Componentes (El Stack Tecnológico)

A. Capa de Cliente (Frontend - React)

"La interfaz de usuario está construida con **React.js**. Esta capa es responsable de todo lo que el usuario ve y con lo que interactúa.

- **Renderizado Dinámico:** Se encarga de actualizar el DOM de manera eficiente sin necesidad de recargar la página completa, mejorando la experiencia de usuario (UX).
- **Consumo de API:** Actúa como cliente HTTP, enviando solicitudes asíncronas al servidor para obtener o persistir información."

B. Capa de Servidor (Backend - Node.js & Express)

"El núcleo lógico del sistema se ejecuta sobre **Node.js**, utilizando el framework **Express**. Esta capa actúa como una **API RESTful** que centraliza la reglas de negocio.

- **Enrutamiento:** Express gestiona las rutas (endpoints) que reciben las peticiones del frontend.
- **Intermediario:** Procesa los datos entrantes, realiza validaciones de seguridad y coordina la comunicación con la base de datos antes de enviar una respuesta al cliente."

Interacción con la Base de Datos (Driver) "La comunicación entre el servidor Node.js y la base de datos MySQL se gestiona a través de la librería **mysql2**. Este cliente fue seleccionado por su alto rendimiento y compatibilidad con el protocolo estándar de MySQL.

Su implementación permite:

- **Seguridad:** Uso de **sentencias preparadas (prepared statements)**, lo que mitiga vulnerabilidades críticas como la inyección SQL al separar la estructura de la consulta de los datos.
- **Eficiencia:** Gestión de conexiones mediante un **Connection Pool** (grupo de conexiones), optimizando el uso de recursos del servidor y reduciendo la latencia en aplicaciones con múltiples usuarios concurrentes."

C. Capa de Datos (Database - MySQL)

"Para la persistencia de la información se utiliza **MySQL**, un sistema de gestión de bases de datos relacional (RDBMS).

Integridad Referencial: Almacena los datos en tablas estructuradas y normalizadas, asegurando la consistencia y las relaciones entre las distintas entidades del negocio (como Usuarios, Productos, Pedidos, etc.)."

9.6.4. Flujo de Información (Cómo se conectan)

"La comunicación entre el Frontend (React) y el Backend (Express) se realiza a través del protocolo **HTTP**, utilizando **JSON** como formato estándar de intercambio de datos. El servidor no mantiene estado de la interfaz (stateless), lo que significa que cada petición contiene toda la información necesaria para ser procesada, garantizando una arquitectura escalable y modular."

10 Plan de pruebas

10.1 Introducción y Objetivos

"El propósito de este Plan de Pruebas es validar la funcionalidad, fiabilidad y desempeño del sistema SOFTVET. Se busca identificar y corregir errores antes del despliegue final, asegurando que los módulos críticos (autenticación, gestión de clientes/pacientes, historial médico) cumplan con los requerimientos especificados."

10.2 Alcance de las Pruebas (Scope)

"El alcance de las pruebas abarca las siguientes áreas funcionales:

- **Módulo de Autenticación:** Login, Registro de personal, Recuperación de credenciales.
- **Gestión de Usuarios y Roles:** CRUD completo de administradores y veterinarios.
- **Funcionalidades del Negocio:** Registro de mascotas, gestión de historias clínicas y agendamiento de turnos.
- **Validaciones:** Verificación de formularios en el Frontend y manejo de errores.
- **API REST:** Respuestas del servidor y códigos de estado HTTP (200, 404, 500).

Exclusiones:

No se realizaron pruebas de carga masiva (stress testing) ni pruebas de seguridad avanzada (pentesting) para esta versión del prototipo."

10.3 Entorno de Pruebas

"Las pruebas fueron ejecutadas bajo la siguiente configuración técnica:

- **Navegadores:** Google Chrome (v120+), Mozilla Firefox.
- **Sistema Operativo:** Windows 10/11 (Entorno Local).
- **Base de Datos:** MySQL Server.
- **Servidor de Aplicación:** Node.js v18+."

10.4 Estrategia y Herramientas

"Se aplicó una metodología de pruebas manuales y verificación de servicios:

- **Pruebas de API:** Se utilizó **Postman** para verificar los endpoints del Backend, asegurando que la estructura JSON de respuesta y los códigos de estado fueran correctos.
- **Pruebas de Interfaz:** Se realizaron pruebas de usabilidad y flujo de navegación directamente en el navegador (Pruebas de Caja Negra)."

10.5 Casos de Prueba (Muestra Representativa)

ID	Caso de Prueba	Pasos Realizados	Resultado Esperado	Estado
CP-01	Login Correcto	Ingresar credenciales válidas (Vet/Admin) y dar clic en "Entrar".	El sistema redirige al Dashboard principal.	Exitoso
CP-02	Login Incorrecto	Ingresar una contraseña errónea intencionalmente.	El sistema muestra alerta roja: "Credenciales inválidas".	Exitoso
CP-03	Campos Vacíos	Intentar guardar un formulario (ej: Nueva Mascota) dejando campos obligatorios en blanco.	El sistema impide el envío y resalta los campos faltantes.	Exitoso
CP-04	Registrar Mascota	Completar ficha de paciente y guardar.	Aparece mensaje "Guardado con éxito" y se actualiza el listado.	Exitoso
CP-05	Conexión API	Verificar carga de listados (Clientes/Turnos).	Los datos se recuperan correctamente de la Base de Datos.	Exitoso

10. RESULTADOS ESPERADOS

10.1 Resultados y Beneficios

El sistema **SoftVet** permitirá digitalizar por completo las operaciones clínicas y administrativas del centro veterinario, eliminando la dependencia de fichas de papel y reduciendo en un **70% el tiempo dedicado a tareas de gestión manual**.

Con la centralización de datos en una base relacional, el personal médico podrá acceder de forma inmediata a la **Historia Clínica** de los pacientes, mejorando la precisión en los tratamientos y la calidad de atención. Asimismo, la automatización de la agenda de turnos resuelve la problemática detectada de pérdidas de citas por errores de coordinación.

La incorporación de reportes automáticos ofrece una gestión más moderna y profesional, fortaleciendo el vínculo con los propietarios y garantizando un control riguroso sobre el inventario de medicamentos e insumos críticos.

10.2 Resultados Técnicos Alcanzados

El desarrollo del sistema permite obtener los siguientes resultados concretos:

- **Sistema Full-Stack funcional:** Implementación de una arquitectura moderna con React y Node.js.
- **Gestión Clínica Digital:** Módulo operativo de Historias Clínicas y seguimiento de pacientes.
- **Seguridad Avanzada:** Control de acceso y protección de datos sensibles mediante JWT.
- **Control de Stock Inteligente:** Automatización de existencias con alertas de stock mínimo.
- **Automatización de Comunicación:** Integración de notificaciones automáticas para turnos.
- **Documentación Profesional:** Carpeta técnica y manuales conformes a estándares académicos universitarios.

10.3 Impacto Académico

- **Integración Interdisciplinaria:** Articulación de conocimientos de programación, diseño de bases de datos y **Análisis de Datos**.
- **Metodologías Ágiles:** Aplicación efectiva de SCRUM en un entorno de desarrollo colaborativo (Git/GitHub).
- **Arquitectura de Software:** Desarrollo de competencias en la construcción de aplicaciones cliente-servidor escalables.
- **Modelado de Datos Real:** Resolución de problemas de persistencia e integridad en un dominio complejo como el de la salud animal.

10.4 Impacto Institucional y Profesional

- **Solución de Mercado:** Proyecto demostrativo de una herramienta con potencial de implementación real en PyMEs locales.
- **Referencia Técnica:** Ejemplo práctico de un sistema completo que integra tecnologías modernas (React, Express, MySQL).
- **Escalabilidad:** Base sólida para futuras expansiones, como la integración de facturación electrónica o telemedicina veterinaria.

11. REFERENCIAS

- **Express.js.** (s.f.). *Express: Infraestructura web rápida, minimalista y flexible para Node.js*. Recuperado el 28 de abril de 2026, de <https://expressjs.com/es/>
- **GitHub.** (s.f.). *GitHub Documentation*. Recuperado el 28 de abril de 2026, de <https://docs.github.com/es>
- **Jones, M., Bradley, J., & Sakimura, N.** (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc7519>
- **Meta Platforms, Inc.** (s.f.). *React: Una biblioteca de JavaScript para construir interfaces de usuario*. Recuperado el 28 de abril de 2026, de <https://es.react.dev/>
- **OpenJS Foundation.** (s.f.). *Documentación de Node.js*. Recuperado el 28 de abril de 2026, de <https://nodejs.org/es/docs/>
- **Oracle Corporation.** (s.f.). *MySQL 8.0 Reference Manual*. Recuperado el 28 de abril de 2026, de <https://dev.mysql.com/doc/refman/8.0/en/>
- **Postman.** (s.f.). *Postman API Platform*. Recuperado el 28 de abril de 2026, de <https://www.postman.com/>
- **Schwaber, K., & Sutherland, J.** (2020). *La Guía de Scrum: La Guía Definitiva de Scrum: Las Reglas del Juego*. Scrum.org. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf>
- **Tailwind Labs.** (s.f.). *Tailwind CSS Documentation*. Recuperado el 28 de abril de 2026, de <https://tailwindcss.com/docs>